# 嵌入式系统原理

## The Principle of Embedded System



合肥工业大学·计算机与信息学院

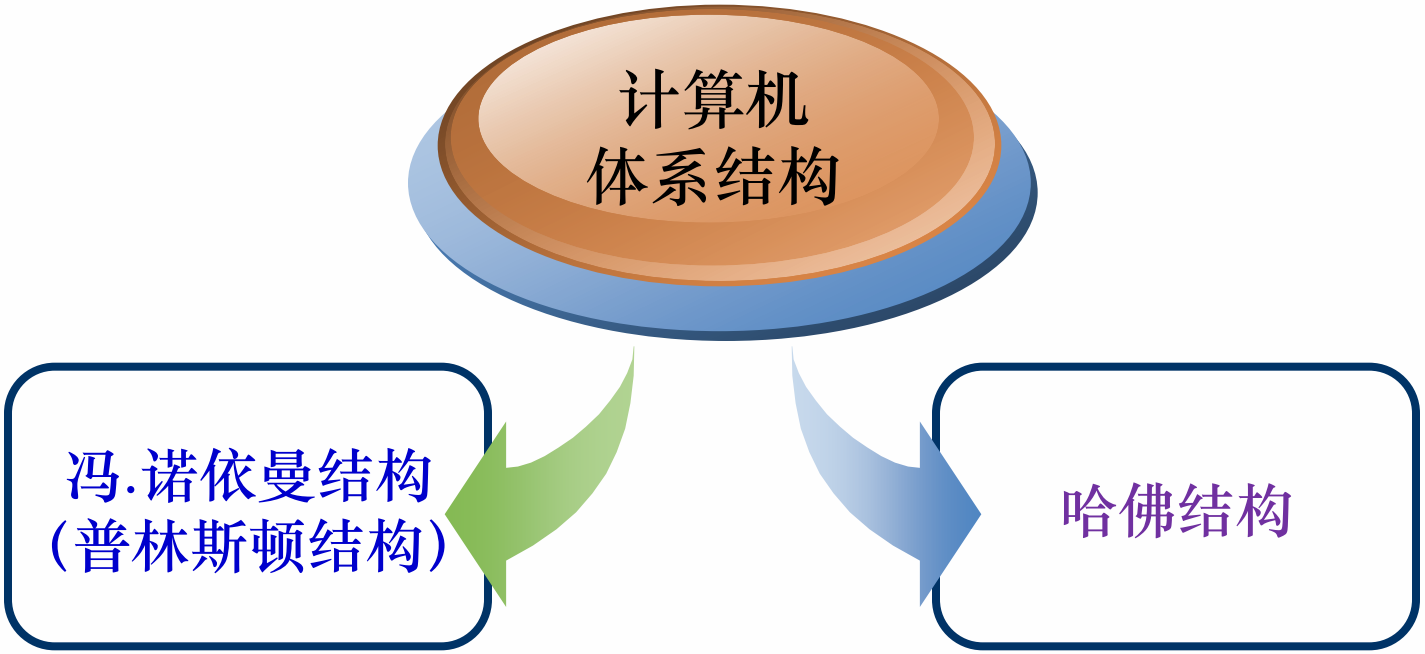






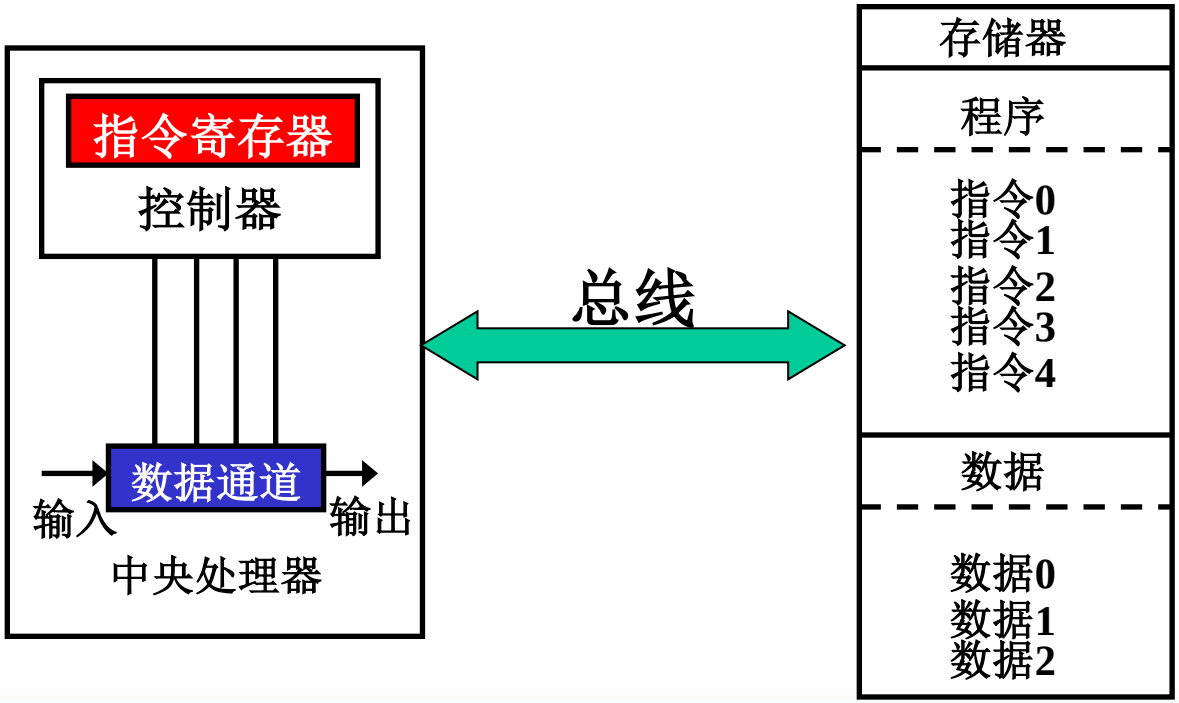
- 2.1 微处理器相关概念
- 2.2 ARM微处理器概述
- 2.3 ARM微处理器结构
- 2.4 ARM微处理器运行模式
- 2.5 ARM微处理器工作状态
- 2.6 ARM微处理器寄存器组织
- 2.7 ARM微处理器存储器格式
- 2.8 ARM微处理器异常处理

*Computer Architecture* (也称为计算机系统结构)，指计算机的机器语言程序员或编译程序编写者所看到的外特性，即计算机的概念性结构和功能特性。



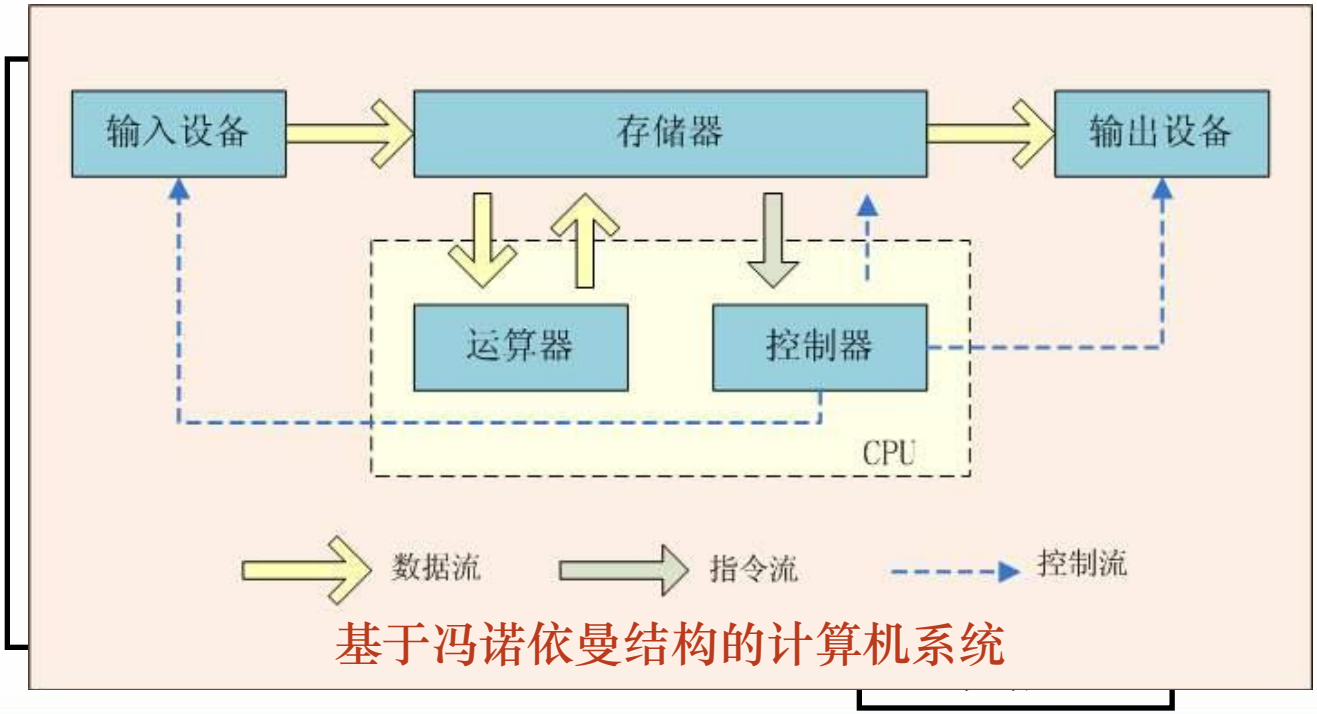
# 冯·诺依曼结构

➢ 核心思想：将程序（指令序列的集合）和数据存放在同一存储器的不同地址；顺序执行指令；执行过程：取指令(or数据)→分析指令→执行指令。



# 冯·诺依曼结构

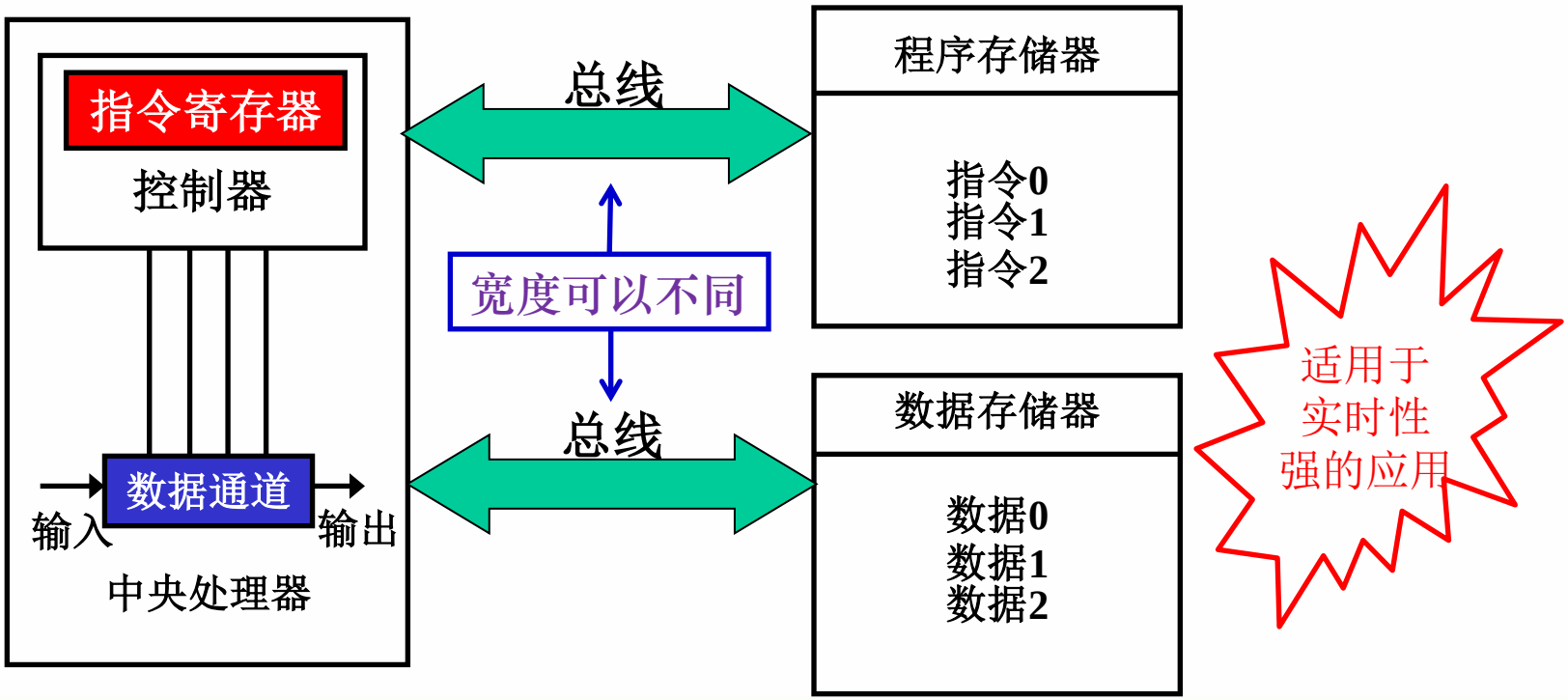
➢ 核心思想：将程序（指令序列的集合）和数据存放在同一存储器的不同地址；顺序执行指令；执行过程：取指令(or数据)→分析指令→执行指令。

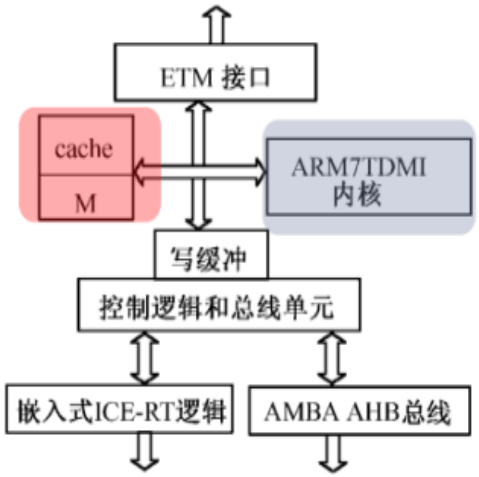



基于冯诺依曼结构的计算机系统

# 哈佛结构

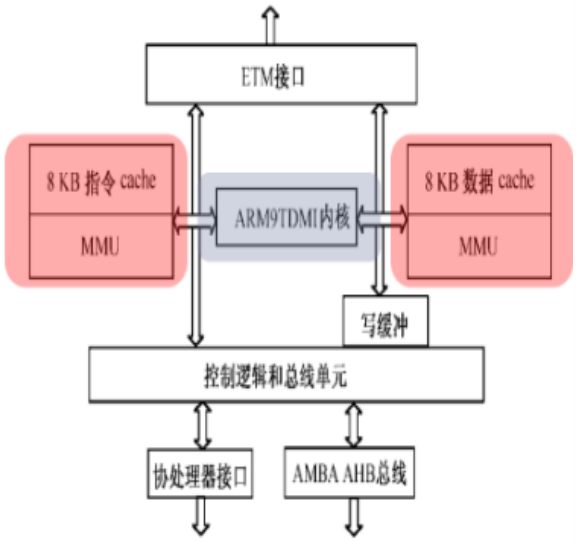
➢ 核心思想：将程序和数据存放在不同的存储器中；并行执行指令；执行过程：取指令(and数据)→分析指令→执行指令。






ARM720T

冯诺依曼




ARM922T

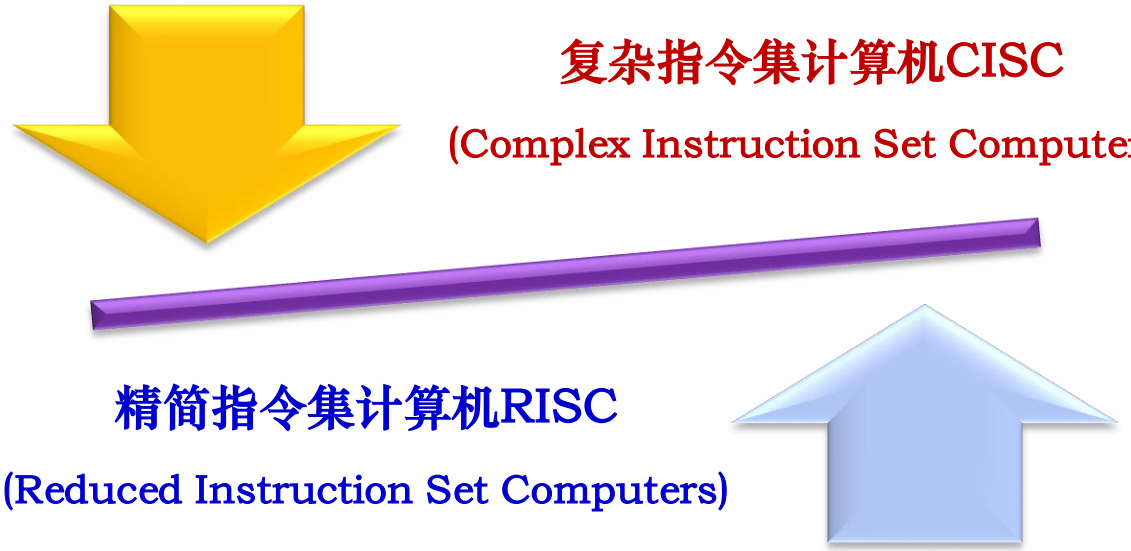
哈佛

@51CTO博客






*Instruction System* (也称为指令集，机器语言)，指计算机所能执行的全部指令(功能)的集合。从系统结构角度看，它实现了软件和硬件的交互联系，是表征计算机性能的重要因素。



## CISC：复杂指令集计算机

- 设计目标：用最少的机器语言指令来完成所需的计算任务。
  - ✓包括大量具有复杂功能的指令和丰富的寻址方式。
  - ✓8/2原则：常用指令仅占指令总数的20%，但在程序中出现的频度却占80%。
- 大多数程序只使用少量的指令就能够运行。
- CPU包括丰富的单元电路→功能强、面积大、价格高、功耗大→与嵌入式系统的要求相悖。

举例：a*=b → MUL ADDRA, ADDRB

## RISC：精简指令集计算机

➢设计目标：借助一些可以在单个CPU周期完成的指令，以降低CPU的复杂度，将程序的复杂性交给编译器。

- ✓只包含最有用的指令,只提供简单的操作。
- ✓确保数据通道快速执行每一条指令。

➢Load-Store结构：CPU只处理寄存器中的数据，使用Load/Store指令来完成数据在寄存器和外部存储器之间的传送。

举例：a*=b

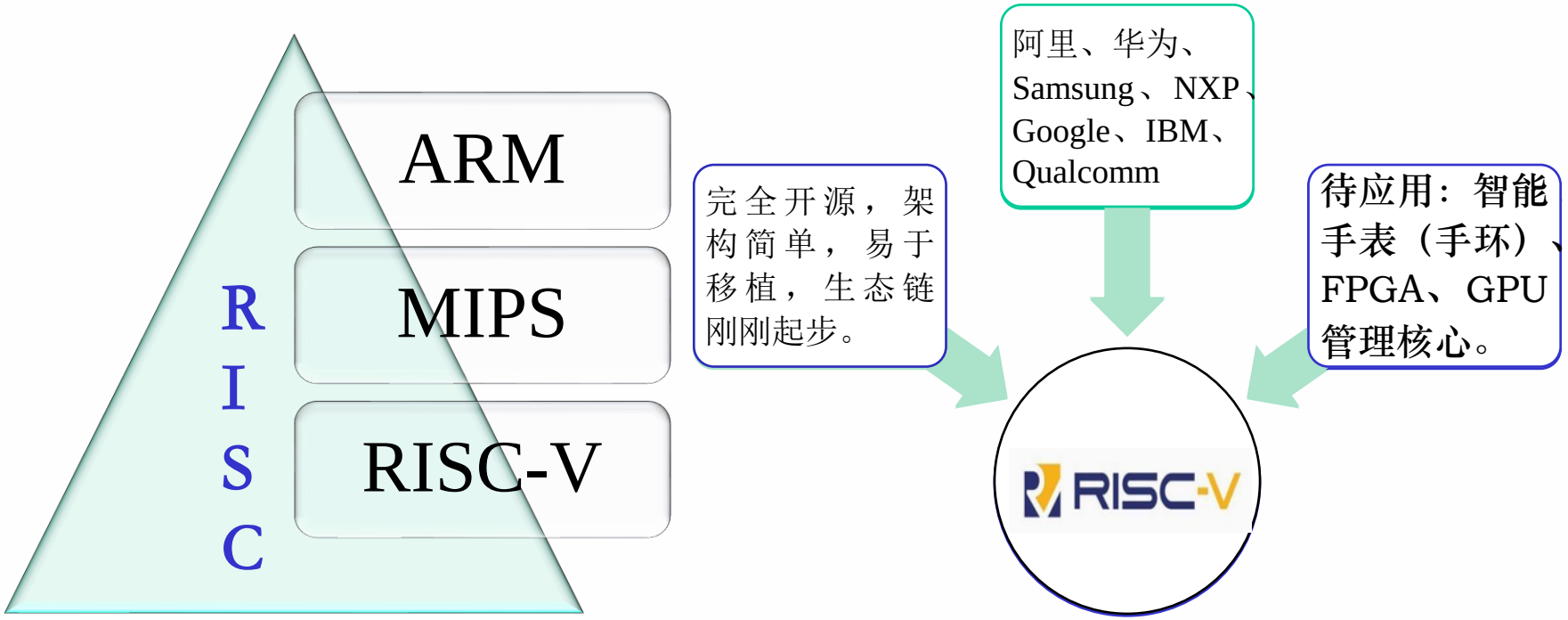
↓

| | (单CPU周期指令) |
|---|---|
| LOAD A, ADDRA; | LOAD B, ADDRB; |
| MUL A, B; | STORE ADDRA, A |

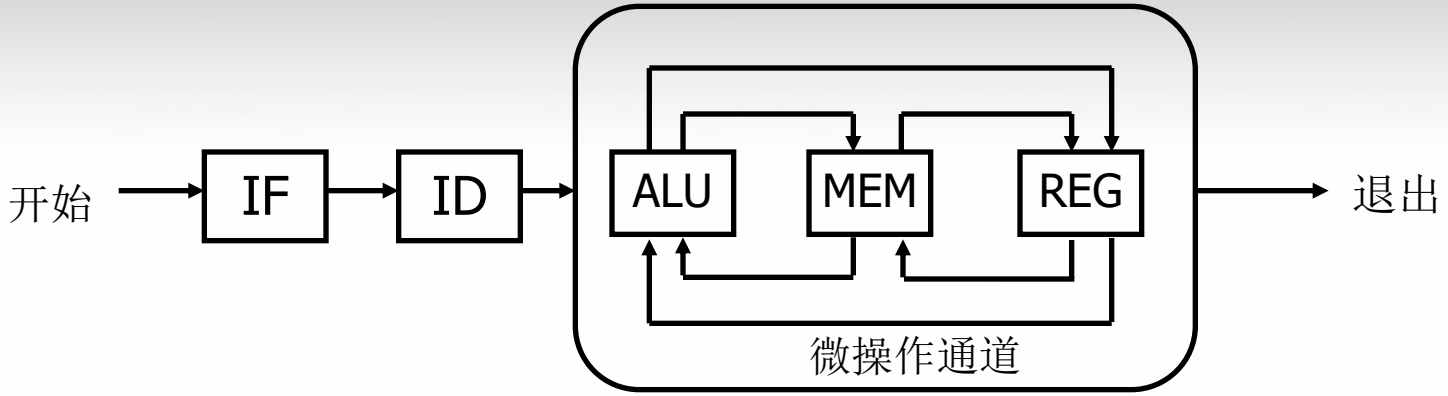
## RISC：精简指令集计算机

- 设计目标：借助一些可以在单个CPU周期完成的指令，以降低CPU的复杂度，将程序的复杂性交给编译器。
  - ✓只包含最有用的指令,只提供简单的操作。
  - ✓确保数据通道快速执行每一条指令。
- Load-Store结构：CPU只处理寄存器中的数据，使用Load/Store指令来完成数据在寄存器和外部存储器之间的传送。
- CPU硬件结构设计变得简单，只包含较少的单元电路，因而面积小、功耗低。

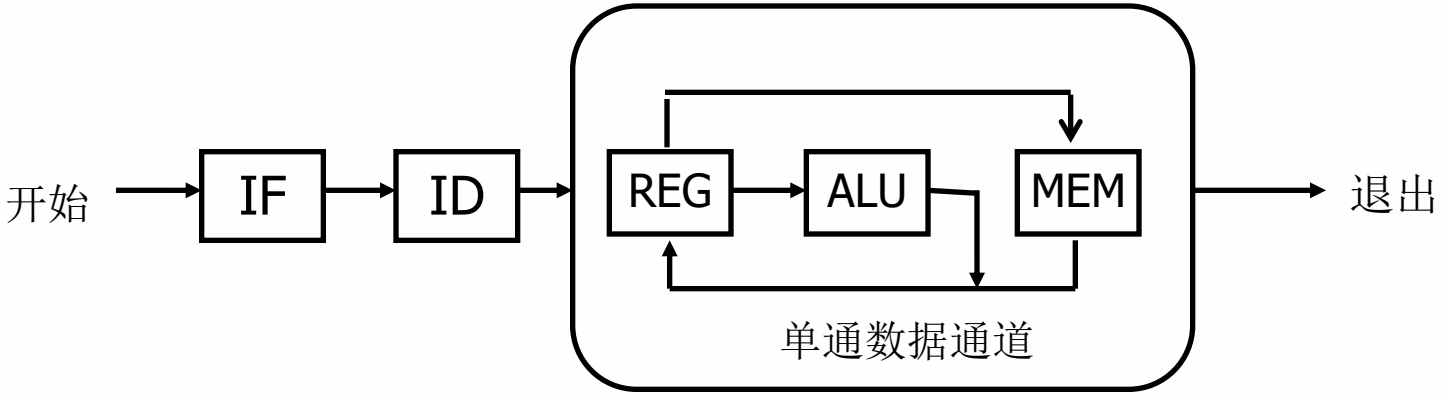
# 主流嵌入式微处理器架构




MIPS= 1. Microprocessor without Interlocked Pipeline Stages
2. Millions of Instructions Per Second




CISC：寻址方式复杂



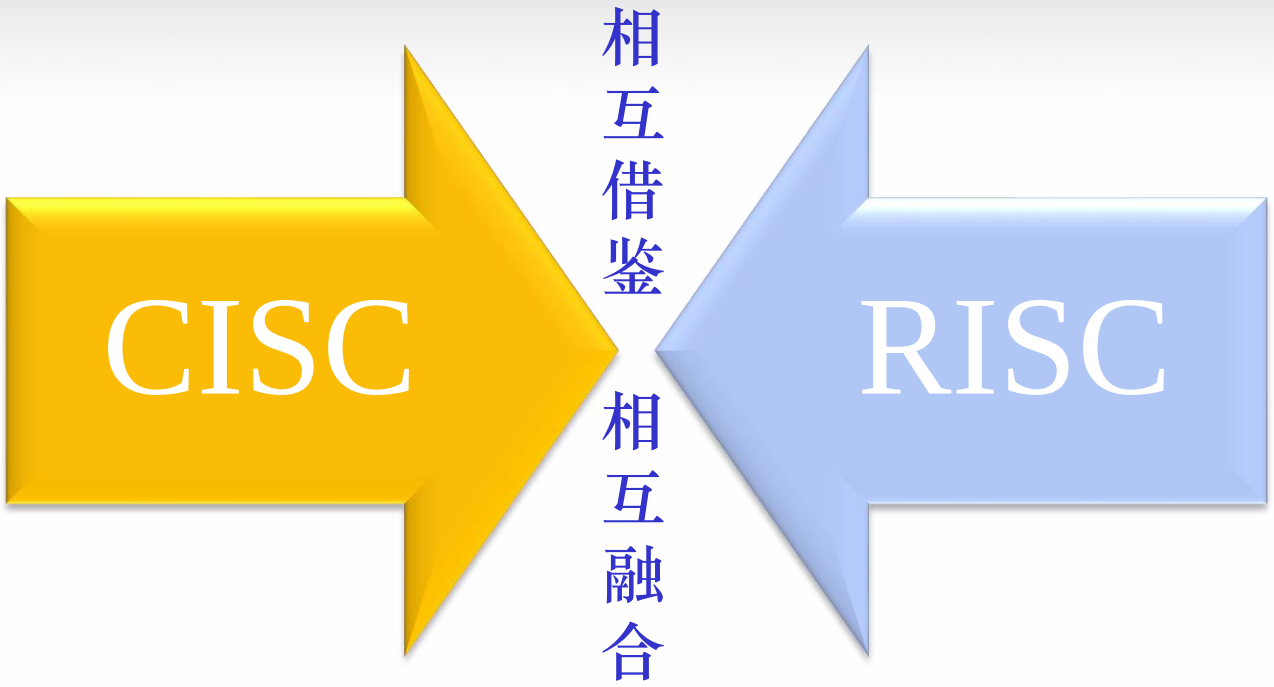

RISC：Load/Store结构







| 类别 | CISC | RISC |
| --- | --- | --- |
| 指令系统 | 指令数量很多 | 较少，通常少于100 |
| 执行时间 | 有些指令执行时间很长，如整块的存储器内容拷贝；或将多个寄存器的内容拷贝到存贮器 | 没有较长执行时间的指令 |
| 编码长度 | 编码长度可变，1-15字节 | 编码长度固定，通常为4个字节 |
| 寻址方式 | 寻址方式多样 | 简单寻址 |
| 操作 | 可以对存储器和寄存器进行算术和逻辑操作 | 只能对寄存器进行算术和逻辑操作，Load/Store体系结构 |
| 编译 | 难以用优化编译器生成高效的目标代码程序 | 采用优化编译技术，生成高效的目标代码程序 |




还有个事实：现在几乎没有纯粹的CISC的计算机。。。现在的x86处理器，内核依然是RISC。。。这个大多数CISC"粉丝"决口不提，或者不知道

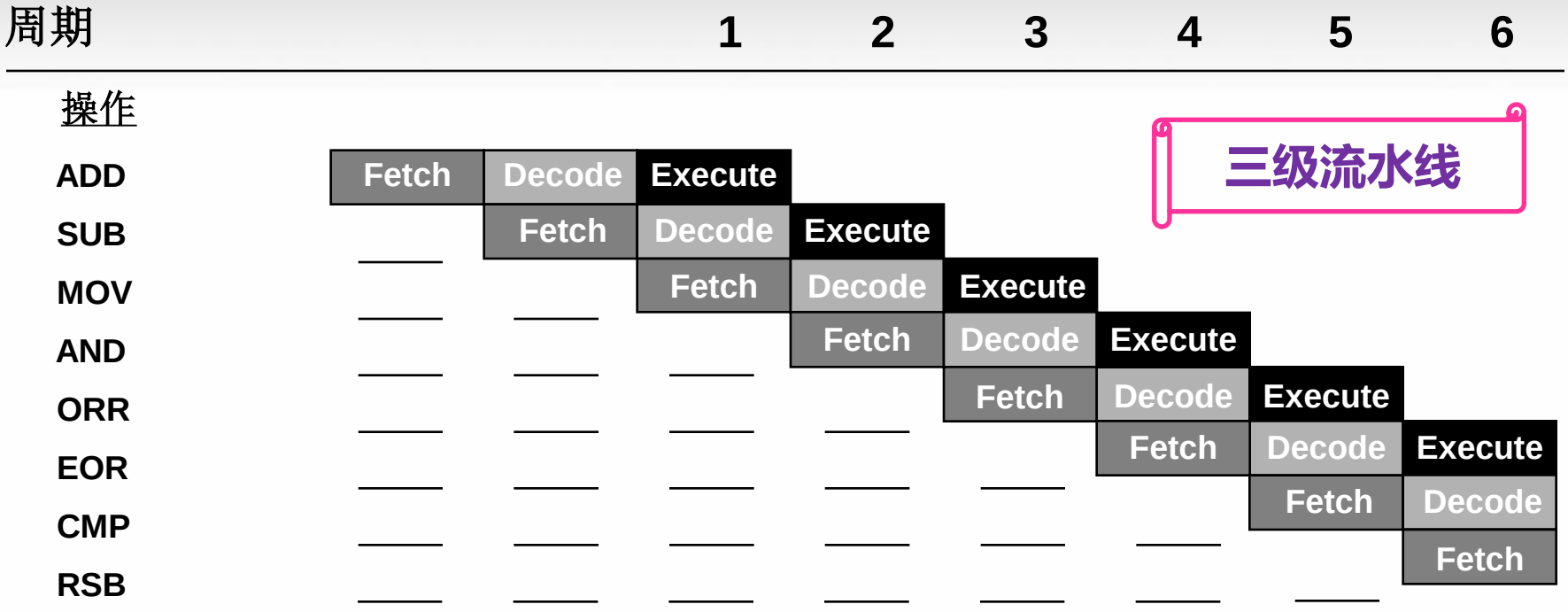
补充一点：其实现在RISC也不是以前那个纯粹的RISC，现在的RISC CPU同样借鉴了部分CISC的特点，包括ARM等在内的RISC。。。

⊕注意：哈佛结构和冯.诺依曼结构主要是指计算机的存储器结构，与指令系统没有严格的对应关系。

- MCS-51是哈佛结构，指令系统是CISC。
- PIC单片机是哈佛结构，指令系统是RISC。
- Apple早期的Macintosh使用IBM CPU(Power)，指令系统是RISC，而现在的使用Intel CPU，指令系统是CISC，它们都是冯.诺依曼结构。

- 流水线(Pipeline)技术：几条指令可以并行执行。
  - 冯诺依曼结构——不区分数据和程序存储器。
  - 为了提高CPU的运行效率。






- 该例中用6个时钟周期执行了6条指令。
- 所有的操作都在寄存器中（单周期执行）
- 指令周期数 (CPI) = 3







- 2.1 微处理器相关概念
- **2.2 ARM微处理器概述**
- 2.3 ARM微处理器结构
- 2.4 ARM微处理器运行模式
- 2.5 ARM微处理器工作状态
- 2.6 ARM微处理器寄存器组织
- 2.7 ARM微处理器存储器格式
- 2.8 ARM微处理器异常处理

# ARM的含义

ARM
- Acorn RISC Machine
- Advanced RISC Machines

三层含义：

① 一个公司的名字:英国知识产权核（IP）设计公司

② 一类微处理器的通称

③ 一种技术的名字（ARM微处理器核）

# ARM的含义

ARM
- Acorn RISC Machine
- Advanced RISC Machines

# ARM的含义

ARM — Advanced RISC Machines

- 一家微处理器行业的知名企业，设计了大量高性能、廉价、耗能低的RISC微处理器。
- 只设计芯片，而不生产。它将技术授权给世界上许多著名的半导体、软件和OEM厂商，并提供服务。

ARM

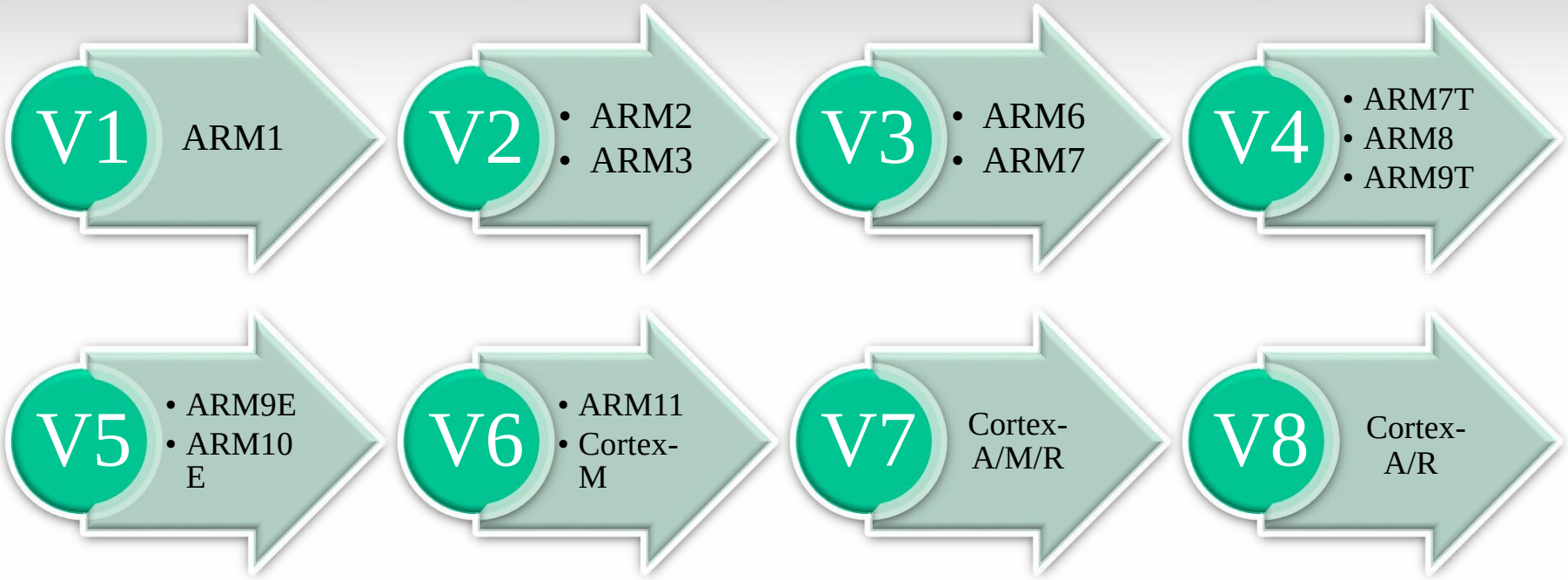
将技术授权给其它芯片厂商

intel IBM LG NEC Newco SONY . . .

形成各具特色的ARM芯片



# ARM内核架构




**ARM V8体系引入 64 位架构支持，包括64位和32位两种执行状态，并支持三个主要指令集。**

# ARM微处理器系列

| ARM微处理器（内核）系列 | 包含类型 |
| --- | --- |
| ARM10E | ARM1020E<br>ARM1022E<br>ARM1026EJ-S |
| ARM11 | ARM1136J-S<br>ARM1136JF-S<br>ARM1156T2(F)-S<br>ARM1176JZ(F)-S<br>ARM11MPCore |
| SecurCore | SC100<br>SC110<br>SC200<br>SC210 |
| 其他合作伙伴产品 | StrongARM<br>Xscale<br>Cortex-M3<br>MBX |

S3C2440



# ARM体系结构版本的命名规则

| ARM Vn | variants | x (variants) |
|---|---|---|

- Vn：对应指令集的版本号。n=1~8
- variants：支持的变种类型
- × (variants)：不支持的变种类型。

# ARM处理器（内核）的命名规则

ARM {x} {y} {z} {T} {D} {M} {I} {E} {J} {F} {-S}

ARM 926EJ-S

x-Family number
- 7: ARM7
- 9: ARM9
- 10: ARM10
- 11: ARM11

y-Memory system
- 2: Cache, MMU, Process ID
- 4: Cache, MPU
- 6: Write buffer, no cache

z-Memory size
- 0: Cache size (4-128KB)
- 2: Reduced cache size
- 6: TCM(Tightly Coupled Memories)

Synthesizable version

Extensions
- E: DSP extension
- J: Jazelle extension
- T: Thumb support
- …

| 标志 | 含义 | 说明 |
| --- | --- | --- |
| T | 支持Thumb指令集 | Thumb指令集版本1：ARMv4T<br>Thumb指令集版本2：ARMv5T<br>Thumb-2：ARMv6T |
| D | 片上调试 | 使处理器能够停止，以响应调试请求 |
| M | 支持长乘法 | 32位乘32位得到64位，32位的乘加得到64位 |
| I | Embedded ICE | 提供片上断点和调试点 |
| E | DSP指令 | 增加了DSP算法处理器指令：16位乘加指令，饱和的带符号数的加减法，双字数据操作，cache预取指令 |
| J | Java加速器Jazelle | 提高java代码的运行速度 |
| S | 可综合 | 提供VHDL或Verilog语言设计文件 |

分析：
ARM7TDMI-S

# ARM处理器系列

| ARM处理器系列 | 特　点 | 应用场合 |
| --- | --- | --- |
| **ARM7** | 提供Thumb16位压缩指令集和Embedded ICE软件调试方式，适用于更大规模的SoC设计。 | 应用于多媒体和嵌入式设备，包括Internet设备、网络和调制解调器设备，以及移动电话、PDA等无线设备。 |
| **ARM9** | 兼容ARM7系列；提供更加灵活的设计。 | 主要应用于发动机管理、仪器仪表、安全系统和机顶盒等领域。 |
| **ARM9E** | 含有DSP指令集；在ARM7处理器内核的基础上使用了Jazelle增强技术。 | 应用于下一代无线设备、数字消费品、成像设备、工业控制、存储设备和网络设备等领域。 |
| **ARM10E** | 使用向量浮点(VFP)单元VFP10提供高性能的浮点解决方案，从而极大提高了处理器的整型和浮点运算性能。 | 主要应用于视频游戏机和高性能打印机等场合。 |
| **Xscale** | 提供全性能、高性价比、低功耗的解决方案，支持16位Thumb指令并集成数字信号处理（DSP）指令。 | 主要应用于手提式通讯和消费电子类设备。 |

| Cortex系列处理器 | 特　点 | 应用场合 |
| --- | --- | --- |
| **Cortex-M3** | 采用了纯Thumb2指令的执行方式，具有极高的运算能力和中断响应能力。 | 主要应用于汽车车身系统，工业控制系统和无线网络等对功耗和成本敏感的嵌入式应用领域。 |
| **Cortex-R4** | 具有双发送、预取和分支预测功能、以及低延迟中断系统，可以中断多周期操作而快速进入中断服务程序。双内核可以组成一个带有失效检测逻辑的冗余锁步(lock-step)配置，非常适合安全攸关的系统。 | 主要应用于产量巨大的高级嵌入式应用系统，如硬盘、喷墨式打印以及汽车安全系统等。 |
| **Cortex-R4F** | 加入了代码错误校正(ECC)技术，浮点运算单元(FPU)以及DMA综合配置的能力，增强了处理器在存储器保护单元、缓存、紧密耦合存储器、DMA访问以及调试方面的能力。 | 适用于家电以及电子行业等高端应用领域。 |
| **Cortex-A8** | 运用了增加代码密度和加强性能的技术、支持多媒体及信号处理能力的技术、支持Java和其他文字代码语言的技术。 | 适用于家电以及电子行业等高端应用领域。 |



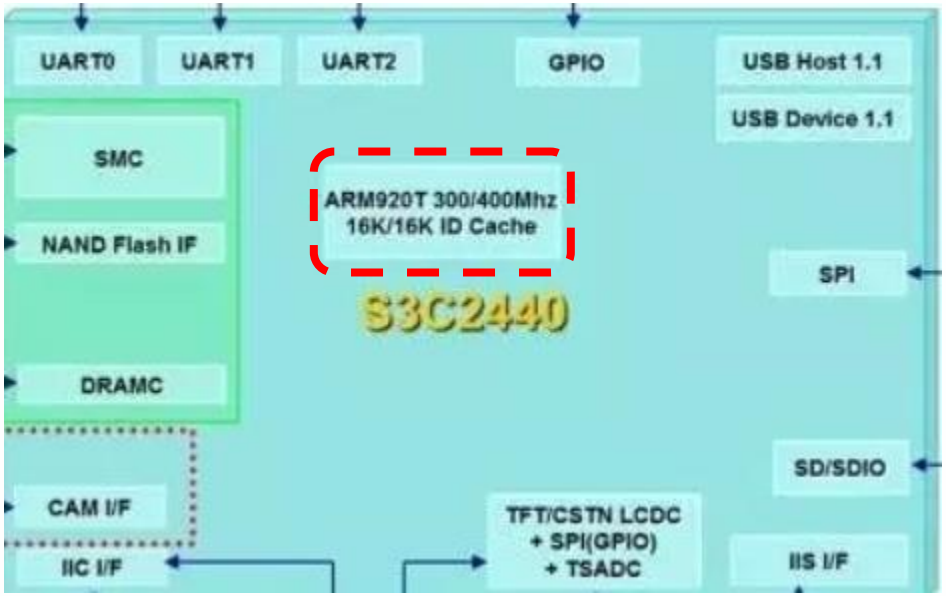





- 2.1 微处理器相关概念
- 2.2 ARM微处理器概述
- **2.3 ARM微处理器结构**
- 2.4 ARM微处理器运行模式
- 2.5 ARM微处理器工作状态
- 2.6 ARM微处理器寄存器组织
- 2.7 ARM微处理器存储器格式
- 2.8 ARM微处理器异常处理

# ARM芯片内部结构

ARM CPU (内核) + 外部设备 (部件) = ARM芯片



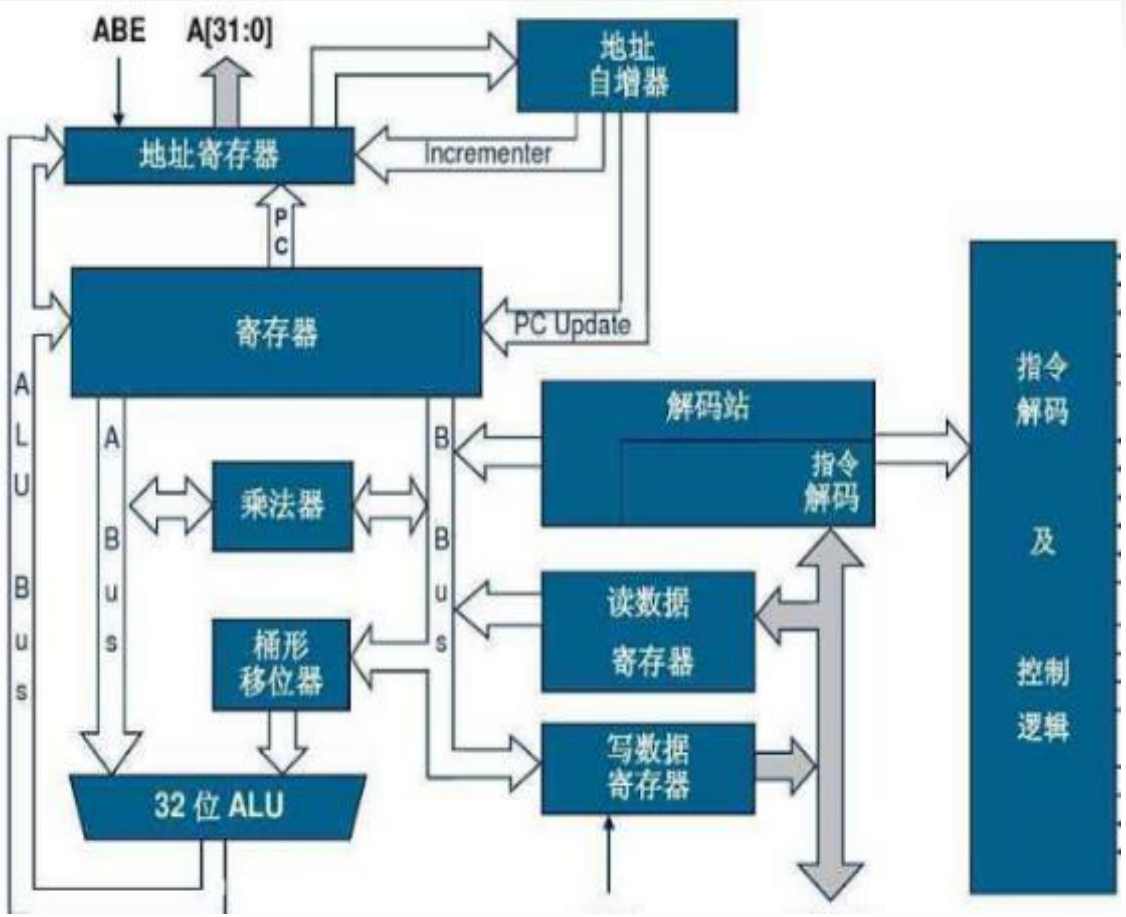



## ARM内核结构

微处理器 = 运算部件 + 控制部件 + 寄存器组 + 总线

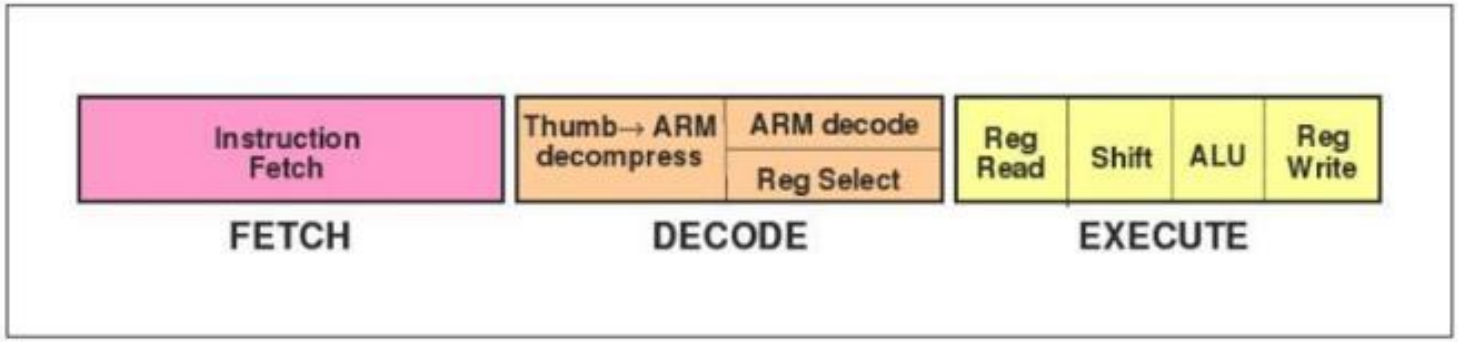
## ARM内核结构

- 包含ALU、桶形移位器、乘法器、浮点部件(可选)、指令译码及控制逻辑、指令流水线、数据/地址寄存器、状态寄存器、总线等。



# ARM7TDMI微处理器

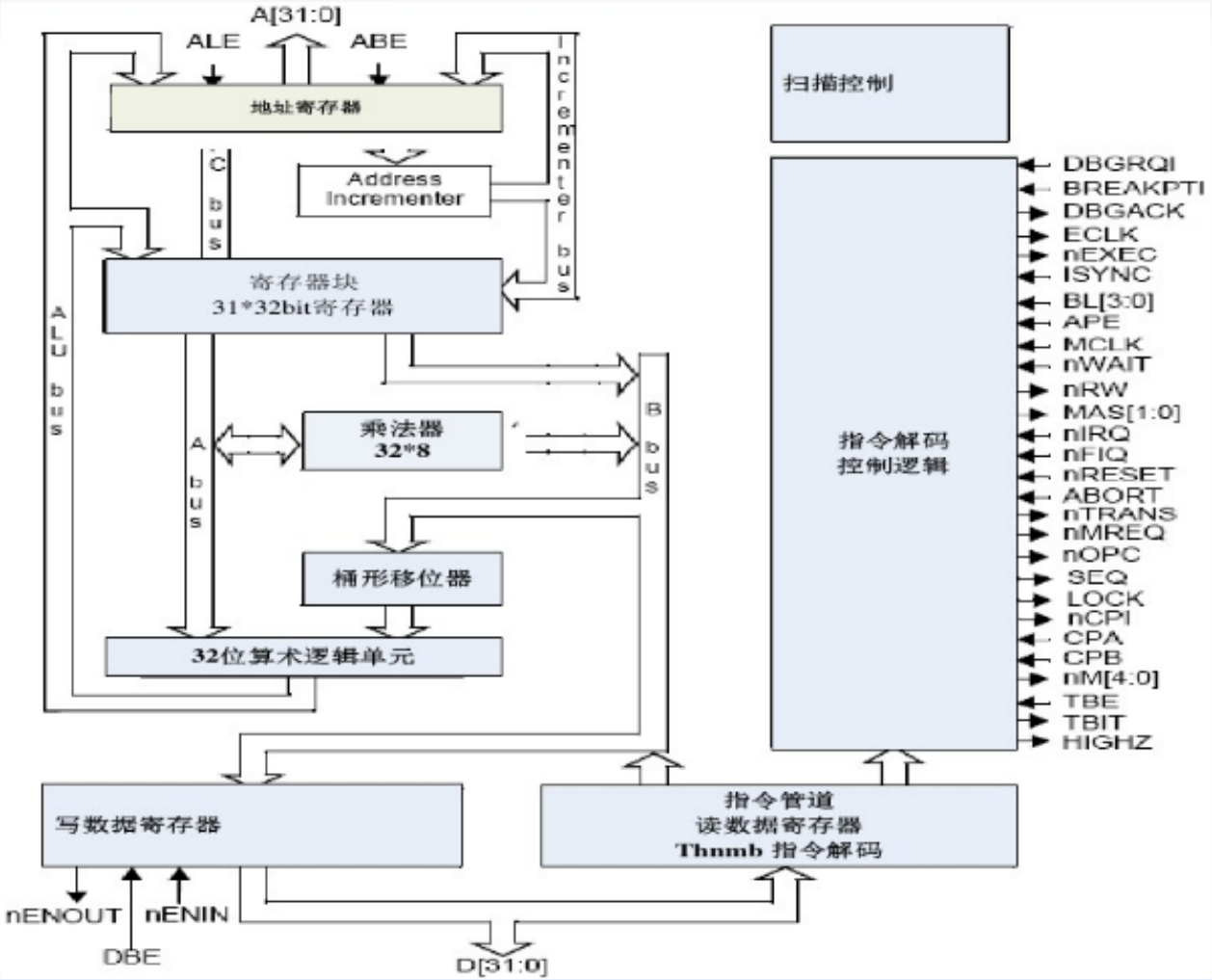
Figure 1 : The ARM7TDMI core and ARM7TDMI-S core pipeline




- 三级流水线：取指令→指令译码→执行指令。
  - ✓ 可提供0.9MIPS/MHz的指令执行速度。
- 性能优异，功耗很低
  - ✓ 高的指令吞吐量
  - ✓ 出色的实时中断响应
  - ✓小的、高性价比的处理器宏单元

# ARM7TDMI微处理器

结构框图



# ARM7TDMI微处理器

- 算术逻辑单元ALU
  - ✓ 实现算术运算和逻辑运算
- 寄存器组
  - ✓ 由31个32位通用寄存器（R0～R30）和6个32位状态寄存器组成，用于保存处理器状态和工作中的数据。
- 对外端口
  - ✓ 2个读端口：用于读2个源操作数
  - ✓ 1个写端口：用于写入目的操作数

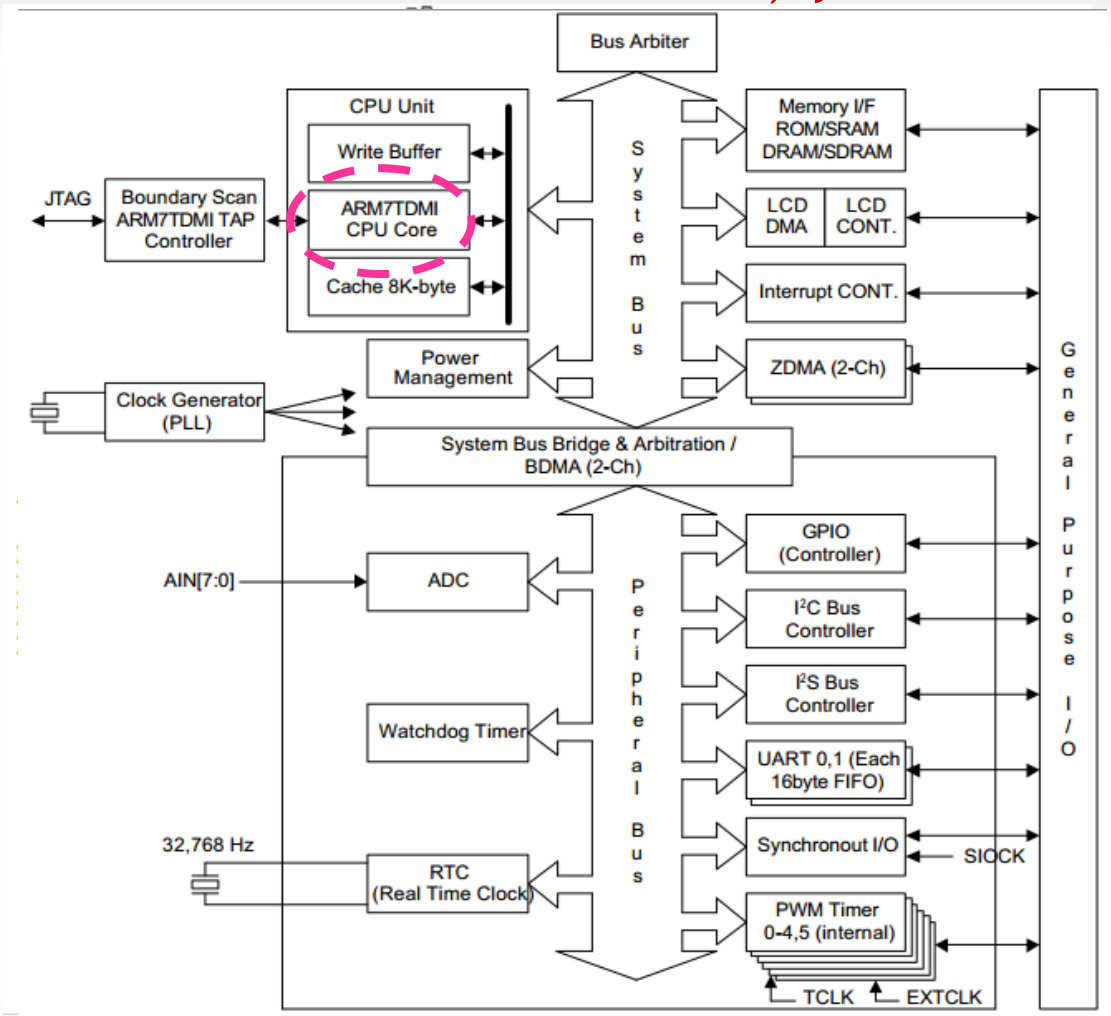
  } 可访问任意寄存器

  - ✓ 1个附加读端口：用于访问程序计算器R15
  - ✓ 1个附加写端口

# ARM7TDMI微处理器

- 桶式移位器
  - ✓ 主要用于完成指令中第2个操作数移位
- 地址寄存器和地址增量器
  - ✓ 选择和保存存储器的地址，并在需要时通过地址增量器生成顺序地址。
- 数据寄存器
  - ✓ 对传送到存储器或从存储器取回的数据进行暂存。
- 控制器
  - ✓ 指令解（译）码器
  - ✓ 相关控制逻辑
  - ✓ ……

# S3C44B0X ARM芯片





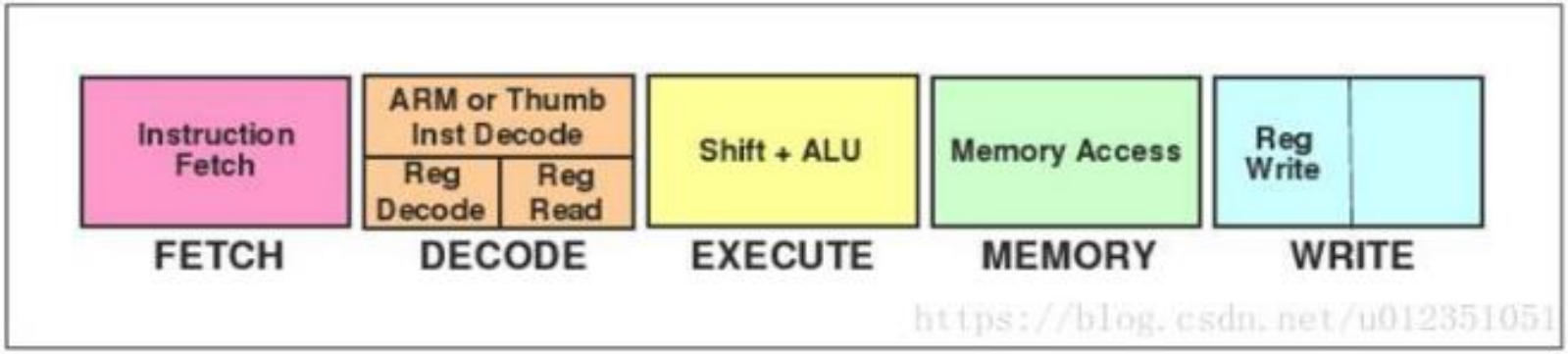


S3C44B0X

内部结构

# ARM9TDMI微处理器

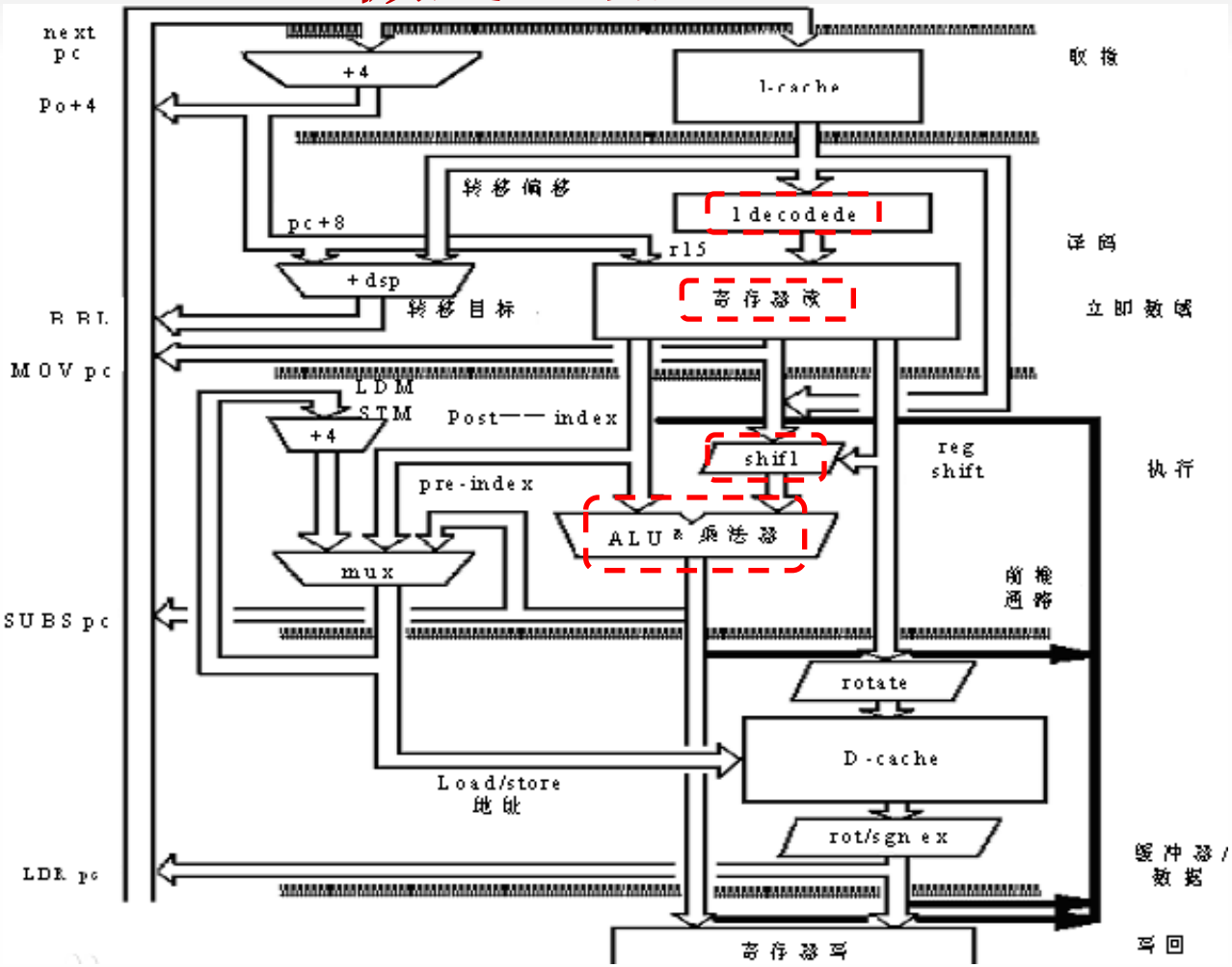
Figure 2 : The ARM9TDMI core pipeline




https://blog.csdn.net/u012351051

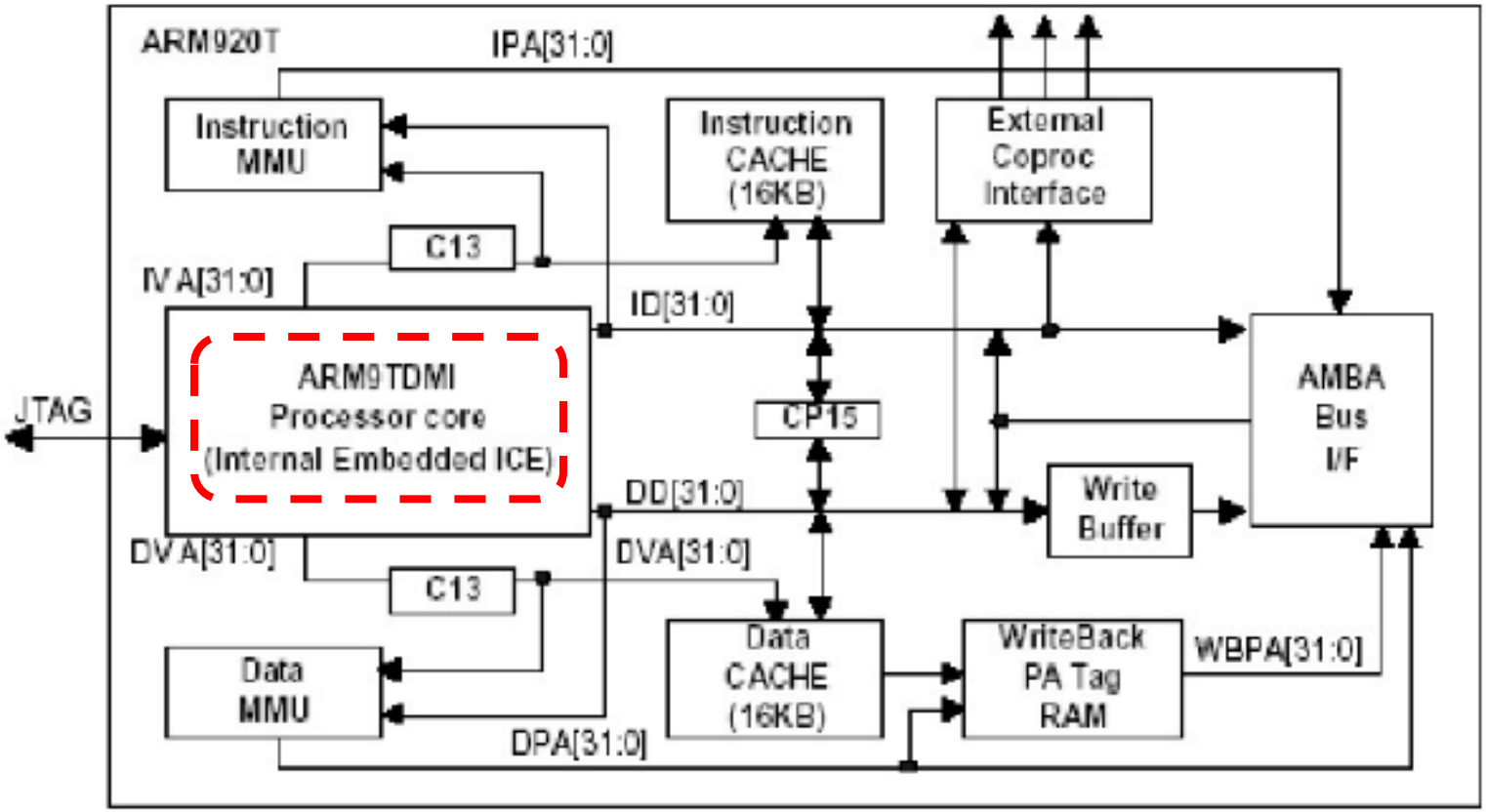
- 五级流水线：取指令→指令译码→执行指令→数据存储器/数据cache访问→寄存器回写。
  - ✓ 可提供300MIPS/MHz的指令执行速度。
- 通过硬件实现Thumb指令解码

## ARM9TDMI微处理器



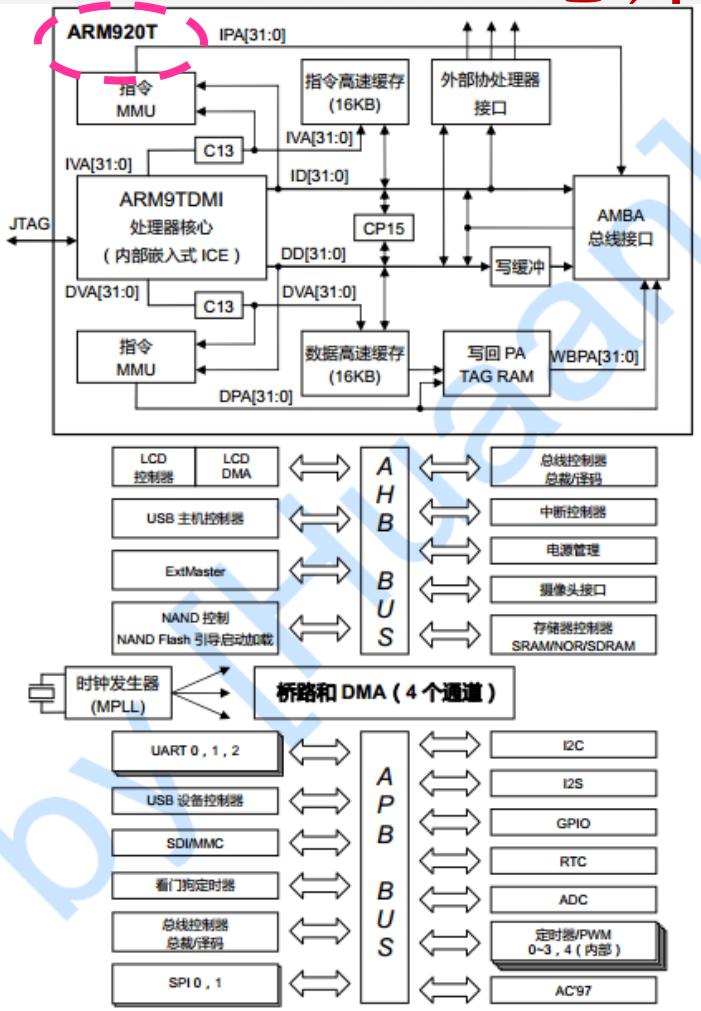

结构框图

# ARM920T微处理器

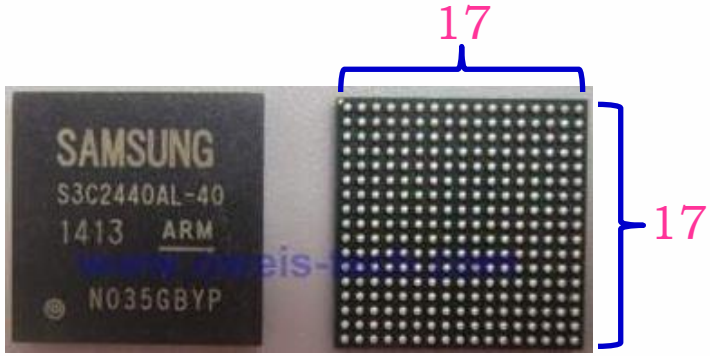



结构框图

# S3C2440 ARM芯片




内部结构




S3C2440

289根引脚








- 2.1 微处理器相关概念
- 2.2 ARM微处理器概述
- 2.3 ARM微处理器结构
- **2.4 ARM微处理器运行模式**
- 2.5 ARM微处理器工作状态
- 2.6 ARM微处理器寄存器组织
- 2.7 ARM微处理器存储器格式
- 2.8 ARM微处理器异常处理







## ARM处理器支持7种运行模式：

| 处理器模式 | 用　途 |
| --- | --- |
| USR（用户模式） | ARM处理器正常的程序执行状态 |
| FIQ（快速中断模式） | 用于高速数据传输或通道处理 |
| IRQ（外部中断模式） | 用户通用的中断处理 |
| SVC（管理模式） | 操作系统使用的保护模式，处理软件中断 |
| ABT（数据访问中止模式） | 当数据或指令预取中止时进入该模式，可用于虚拟存储器和存储器保护 |
| UND（未定义指令模式） | 当未定义的指令执行时进入该模式，可用于支持硬件协处理器的软件仿真 |
| SYS（系统模式） | 运行具有特权的操作系统任务 |

## 用户模式与特权模式

- 除用户模式外，其余的6种模式称为特权模式。

| 模式 |
| --- |
| USR（用户模式） |
| FIQ（快速中断模式） |
| IRQ（外部中断模式） |
| SVC（管理模式） |
| ABT（数据访问中止模式） |
| UND（未定义指令模式） |
| SYS（系统模式） |

特权模式







## 用户模式与特权模式比较

- 特权模式：
  - 程序可以访问所有的系统资源(内部寄存器和片内外设)；
  - 可以任意地进行处理器模式的切换。
- 用户模式：
  - 大多数的应用程序运行在此模式下；
  - 不能访问某些被保护的系统资源；
  - 不能直接切换到其它特权模式，但可以通过产生异常，在异常处理中间接实现模式切换。

## 异常模式

- 除用户和系统模式外，其余的5种模式称为异常模式。
- 用于处理异常（包括中断）和访问受保护的系统资源等情况。

| 模式 |
| --- |
| USR（用户模式） |
| FIQ（快速中断模式） |
| IRQ（外部中断模式） |
| SVC（管理模式） |
| ABT（数据访问中止模式） |
| UND（未定义指令模式） |
| SYS（系统模式） |

异常模式

## 用户模式与系统模式

- 不能由异常直接进入，且使用完全相同的寄存器组。
- 系统模式：特权模式，没有用户模式的限制。
  - OS在该模式下可以访问用户模式的寄存器；
  - 特权任务可以使用该模式访问一些受控的资源。

| 模式 |
| --- |
| USR（用户模式） |
| FIQ（快速中断模式） |
| IRQ（外部中断模式） |
| SVC（管理模式） |
| ABT（数据访问中止模式） |
| UND（未定义指令模式） |
| SYS（系统模式） |

## 模式的切换方法

- 运行模式可以直接或间接地通过程序切换进入。
- 当特定的异常出现时，处理器进入相应的模式。

| 处理器模式 | 备　注 |
| --- | --- |
| USR（用户模式） | 不能直接切换到其它模式 |
| SYS（系统模式） | 与用户模式类似，但具有可以直接切换到其它模式等特权 |
| FIQ（快速中断模式） | FIQ异常响应时进入此模式 |
| IRQ（外部中断模式） | IRQ异常响应时进入此模式 |
| SVC（管理模式） | 系统复位和软件中断响应时进入此模式 |
| ABT（数据访问中止模式） | 在ARM7TDMI中几乎没有用处 |
| UND（未定义指令模式） | 未定义指令异常响应时进入此模式 |







## 启动时的模式转换

管理模式SVC (Supervisor) → 多种特权模式变化 → 用户程序的运行模式

复位后缺省模式

主要完成各模式的堆栈设置，注意不要进入用户模式

一般为用户模式USR



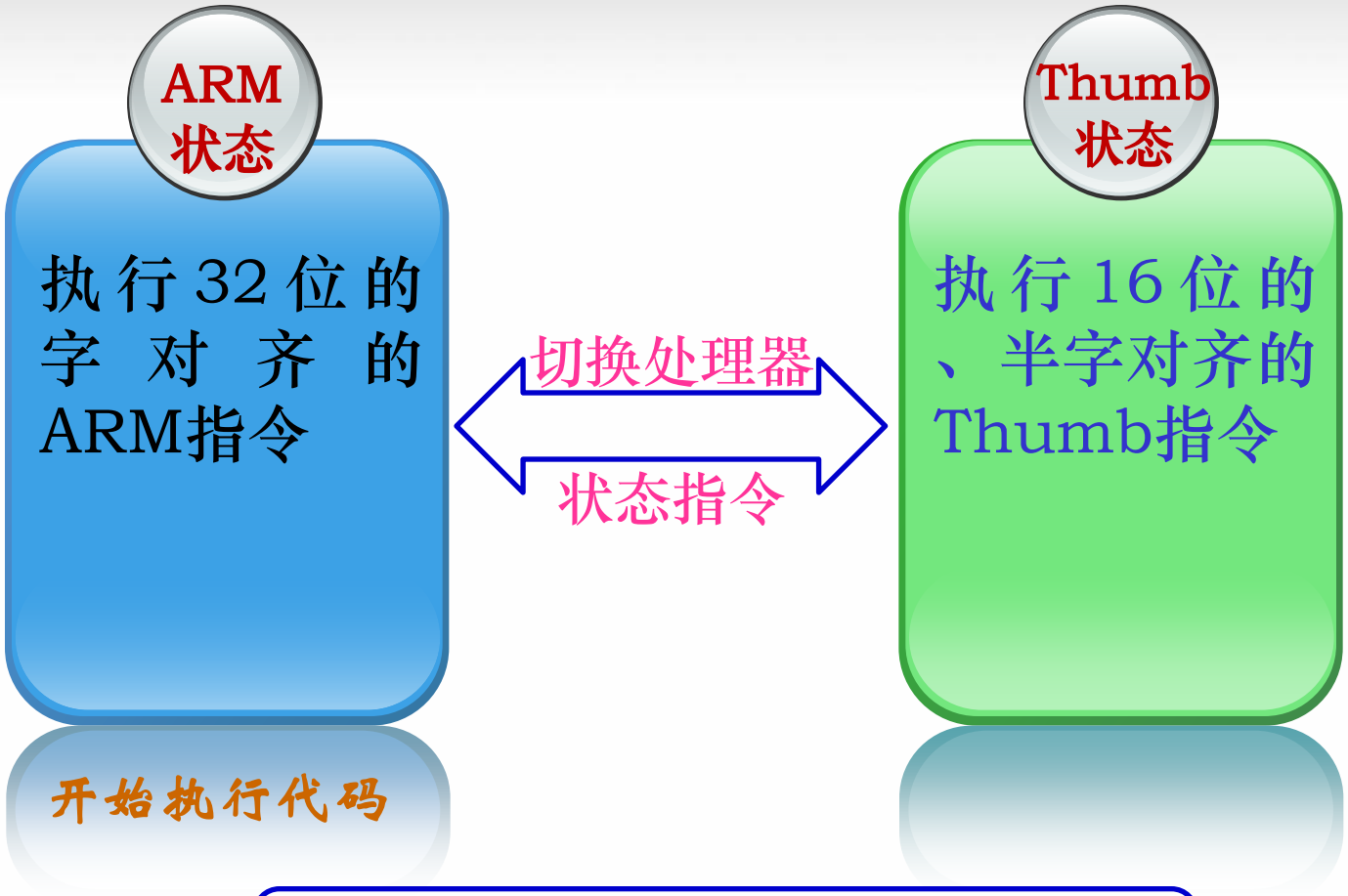





- 2.1 微处理器相关概念
- 2.2 ARM微处理器概述
- 2.3 ARM微处理器结构
- 2.4 ARM微处理器运行模式
- **2.5 ARM微处理器工作状态**
- 2.6 ARM微处理器寄存器组织
- 2.7 ARM微处理器存储器格式
- 2.8 ARM微处理器异常处理

# ARM工作状态






★注意：两种状态之间的切换，不影响处理器的运行模式或寄存器的内容。

# ARM指令集与Thumb指令集

- ARM指令集
  - ✓ 具有固定宽度的32位指令，需要4字节对齐。
  - ✓ 在有限的存储空间中，存放的指令少，代码密度低。

  （注：代码密度指单位存储空间中包含的指令条数。）
- Thumb指令集
  - ✓ 自ARMv4T版架构(ARM7TDMI)开始，为了减少代码量而提出。
  - ✓ 只支持一些通用功能，不是一套完整的指令体系，可看作ARM指令压缩形式的子集。
  - ✓ 必要时仍需要ARM指令集支持。

## 两种指令集的区别

| | ARM指令集 | Thumb指令集 |
|---|---|---|
| 协处理器指令 | 有 | 无 |
| 信号量指令 | 有 | 无 |
| 访问CPSR/SPSR指令 | 有 | 无 |
| 乘加/64位乘法指令 | 有 | 无 |
| 跳转指令 | 三种跳转指令都可以指定条件 | **除B指令外，都是无条件跳转**；跳转范围小 |
| 数据处理指令 | 三个操作数 | 两个操作数；访问寄存器R8~R15受到一定限制 |
| 单寄存器加载/存储指令 | 无限制 | 只能访问寄存器R0~R7 |
| 批量寄存器加载/存储指令 | 无限制 | 只能访问R0~R7的子集 |

# 两种指令集的区别

| | ARM指令集 | Thumb指令集 |
|---|---|---|
| 协处理器指令 | | |
| 信号量指令 | | |
| 访问CPSR/SPSR指令 | | |
| 乘加/64位乘法指令 | | |
| 跳转指令 | | |
| 数据处理指令 | | |
| 单寄存器加载/存储指令 | | |
| 批量寄存器加载/存储指令 | 无限制 | 只能访问R0~R7的子集 |

ARM跳转指令

| 助记符 | 说明 | 操作 | 条件码位置 |
|---|---|---|---|
| B label | 跳转指令 | Pc←label | B{cond} |
| BL label | 带链接的跳转指令 | LR←PC-4, PC←label | BL{cond} |
| BX Rm | 带状态切换的跳转指令 | PC←label,切换处理状态 | BX{cond} |

Thumb跳转指令

| 助记符 | 说明 | 操作 | 条件码位置 |
|---|---|---|---|
| B label | 跳转指令 | PC←label | B{cond} |
| BL label | 带链接的跳转指令 | LR←PC←4,PC←label | 无 |
| BX Rm | 带状态切换的跳转指令 | PC←label 切换处理器状态 | 无 |

## 切换到Thumb状态的方法

- 当寄存器Rm的状态位bit[0]=1时，可以通过执行“BX Rm”指令,使微处理器从ARM状态→Thumb状态。

★例子：从ARM状态切换到Thumb状态：




指令集关系

```
        CODE32
        LDR     R0, =Lable+1
        BX      R0
        CODE16
Lable   MOV     R1, #12
```

程序代码

跳转地址标号

地址最低位为1，表示切换到Thumb状态

执行完BX指令，处理器切换到Thumb状态，开始执行Thumb指令

## 切换到Thumb状态的方法

- 当寄存器Rm的状态位bit[0]=1时，可以通过执行“BX Rm”指令,使微处理器从ARM状态→Thumb状态。
- 当处理器处于Thumb状态时，若发生异常（如IRQ、FIQ、Undef、Abort、SWI等），则异常处理返回时，自动切换到Thumb状态。

## 切换到ARM状态的方法

- 当寄存器Rm的状态位bit[0]=0时，可以通过执行“BX Rm”指令,使微处理器从Thumb状态→ARM状态。
- 在处理器进行异常处理时，默认将处理器切换到ARM状态。

★例子：从Thumb状态切换到ARM状态：




指令集关系

```
        CODE16
        LDR     R0, =Lable
        BX      R0
        CODE32
Lable   MOV     R1, #10
```

程序代码

跳转地址标号

地址最低位为0，表示切换到ARM状态

执行完BX指令，处理器切换到ARM状态，开始执行ARM指令





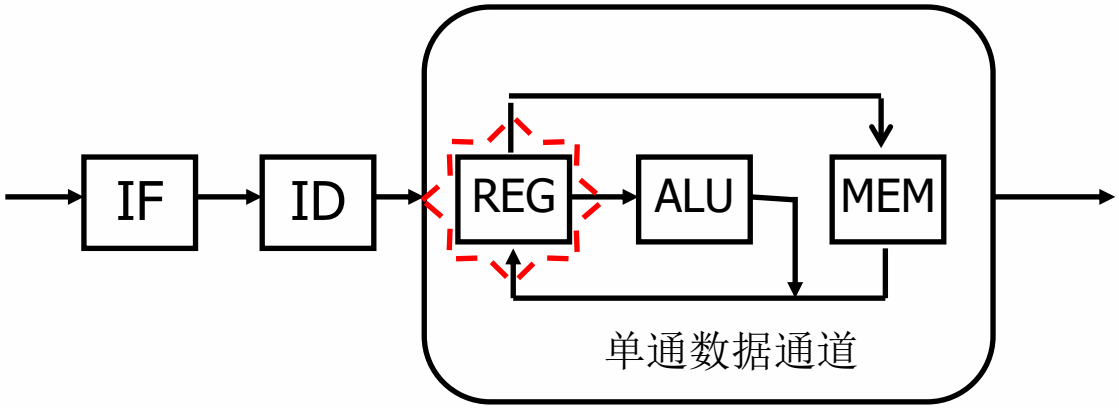



- 2.1 微处理器相关概念
- 2.2 ARM微处理器概述
- 2.3 ARM微处理器结构
- 2.4 ARM微处理器运行模式
- 2.5 ARM微处理器工作状态
- **2.6 ARM微处理器寄存器组织**
- 2.7 ARM微处理器存储器格式
- 2.8 ARM微处理器异常处理

ARM处理器
∈ RISC

Load/Store
结构

Registers
寄存器




RISC：Load/Store结构

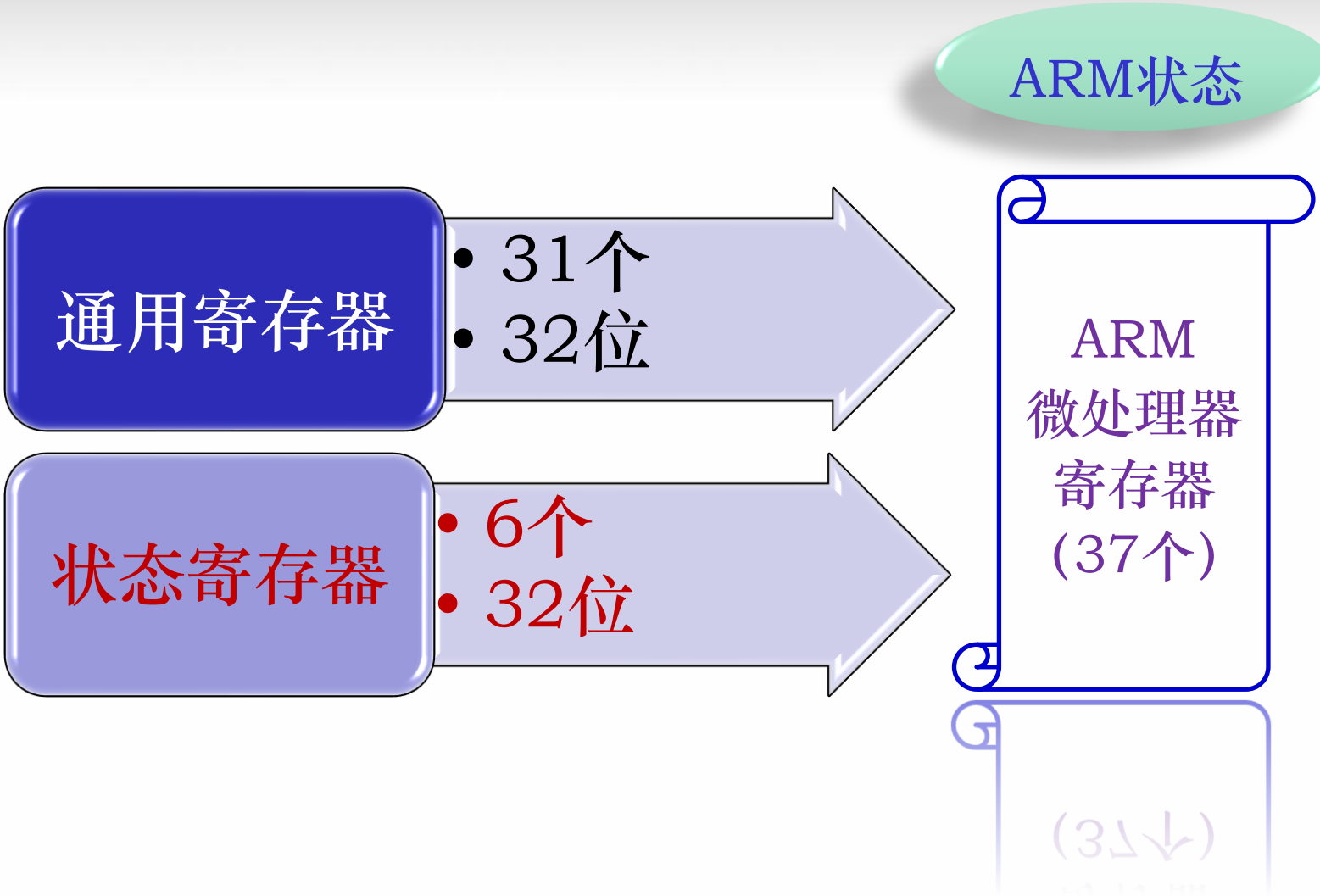
Thumb状态

ARM
微处理器
寄存器
(27个)

★Thumb状态下的寄存器组是ARM状态下寄存器组的一个子集。

ARM状态

ARM
微处理器
寄存器
(37个)

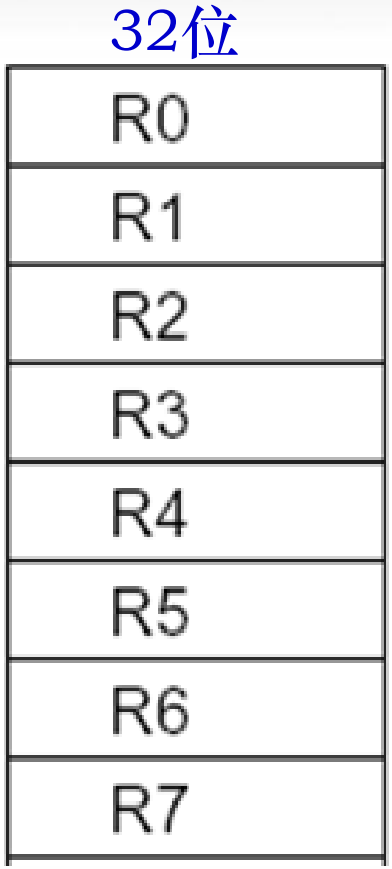

ARM状态
通用寄存器
• 31个
• 32位
状态寄存器
• 6个
• 32位
ARM
微处理器
寄存器
(37个)

# 通用寄存器（31个）

- 未分组寄存器（R0～R7）
  - ✓所有七种运行模式下，都指向相同的物理寄存器。
  - ✓用于保存数据或者地址，未被用作特殊的用途。

（——“货真价实”的通用寄存器）

- ✓ 注意：要特别小心工作模式转换时，可能会造成寄存器中数据的破坏。



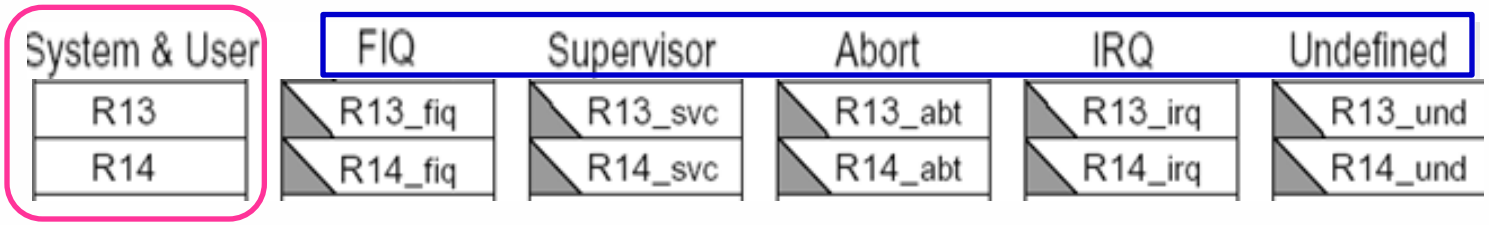
# 通用寄存器（31个）

- 分组寄存器（R8～R14）
  - ✓ 分组寄存器R8～R12
    - 每次访问的物理寄存器与当前的运行模式有关。
    - 每个寄存器对应两个不同的物理寄存器。
      - 当使用快速中断模式时，对应R8_fiq～R12_fiq；
      - 当使用其他模式时，对应R8_usr～R12_usr。

| System & User | FIQ | Supervisor | Abort | IRQ | Undefined |
|---|---|---|---|---|---|
| R8 | R8_fiq | R8 | R8 | R8 | R8 |
| R9 | R9_fiq | R9 | R9 | R9 | R9 |
| R10 | R10_fiq | R10 | R10 | R10 | R10 |
| R11 | R11_fiq | R11 | R11 | R11 | R11 |
| R12 | R12_fiq | R12 | R12 | R12 | R12 |

# 通用寄存器（31个）

- 分组寄存器（R8～R14）（续）
  - ✓ 分组寄存器R13～R14
    - 每个寄存器对应6个不同的物理寄存器。
      - 其中1个是用户模式与系统模式共用，另外5个对应其它五种运行模式。
      - 采用R13_<mode>、R14_<mode>来区分不同的物理寄存器。



## R13—堆栈指针

- 在ARM指令集中，常用作堆栈指针。——习惯用法
- 在Thumb指令集中，某些指令强制性要求使用R13作为堆栈指针。
- 每种运行模式都有自己独立的物理寄存器R13，故在初始化时，需要初始化每种模式对应的R13。

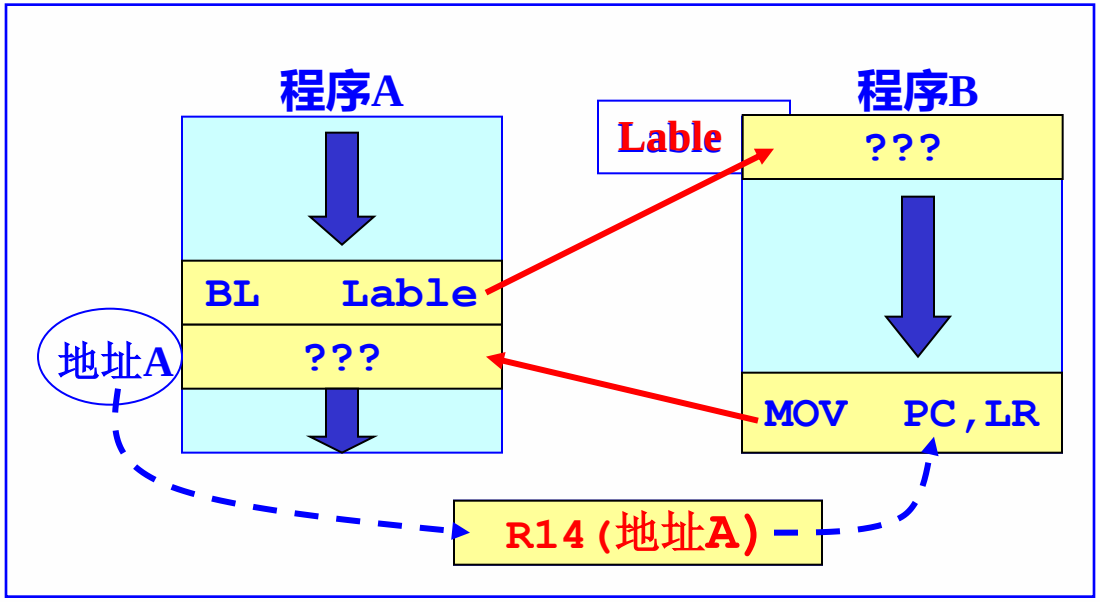
★使用技巧：当程序运行进入异常模式时，可以将需要保护的寄存器内容放入R13所指向的堆栈，而当程序从异常模式返回时，则可以从对应的堆栈中进行恢复。

# R14—子程序链接寄存器

- 又称为链接寄存器LR 。
- 当执行子程序调用指令BL时，可以用于得到程序计数器PC(R15)的备份。
- 其它情况下，用作通用寄存器。
- 两种特殊功能
  - 在每种模式下都可以用于保存子程序的返回地址。
  - 发生异常时，用于保存异常处理后的返回地址，如中断处理等。

# 举例：R14在子程序调用中作用

1.程序A执行过程中调用程序B；
2.程序跳转至标号Lable处，执行程序B。同时硬件将“BL　Lable”指令的下一条指令所在地址存入R14；
3.程序B执行至最后，将R14寄存器的内容放入PC，返回程序A。



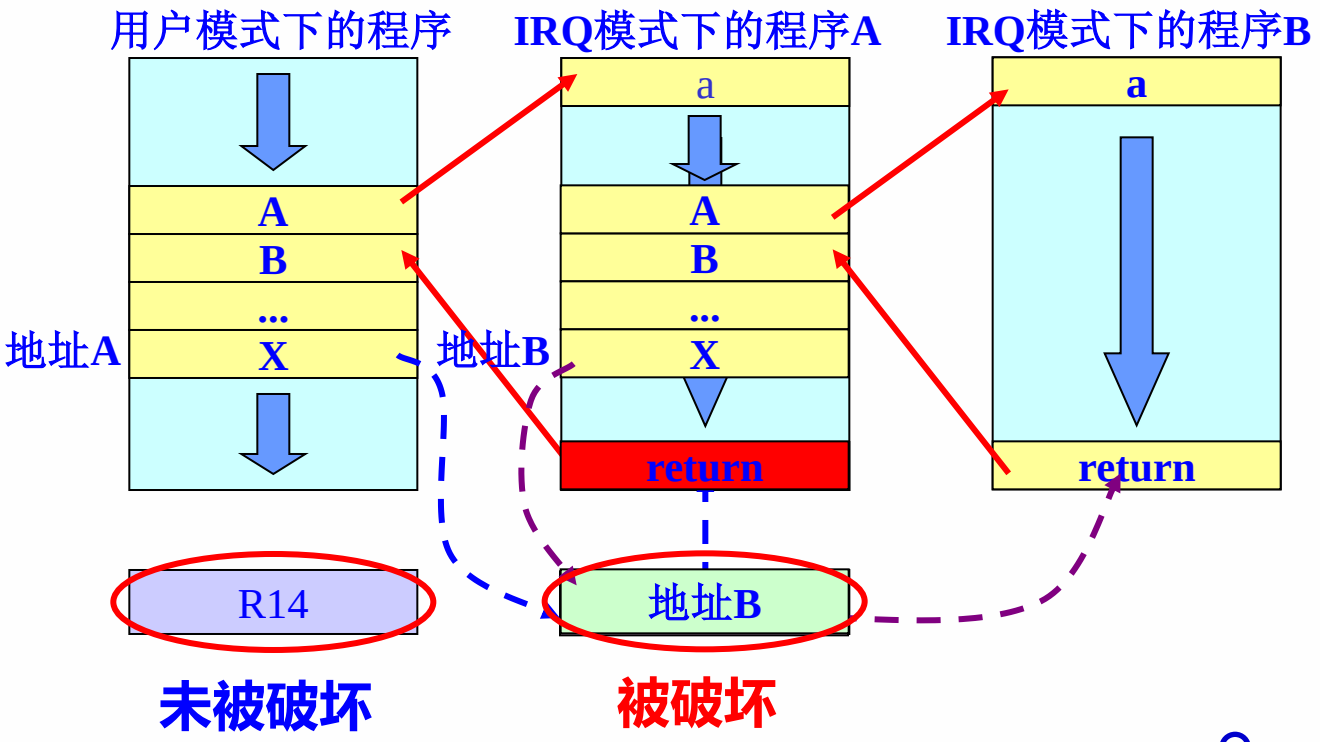
## R14寄存器的注意点

★ 当发生异常嵌套时，这些异常之间可能会发生冲突。

例如：如果在用户模式下执行程序时发生了IRQ中断，用户模式寄存器不会被破坏。但是，如果允许在IRQ模式下的中断处理程序重新使能IRQ中断，并且发生了嵌套的IRQ中断时，则外部中断处理程序保存在R14_irq中的任何值都将被嵌套中断的返回地址所覆盖。

# R14寄存器的注意点

6. 在程序B返回到程序A，然后在返回到用户模式下被中断的程序时，发生错误，将不能正确返回。




★解决方法：发生中断嵌套时，使用堆栈保存R14的值，或者切换到其它处理器模式下。

# 通用寄存器（31个）

- 程序计数器PC（R15）
  - ✓ 所有七种运行模式下，都指向一个物理寄存器。
  - ✓ ARM状态：bit[1:0]=00，bit[31:2]用于存放PC。
    - PC的值=当前正在执行指令的地址值+8。即PC总是指向当前正在执行指令的下两条指令。——原因：ARM采用多级流水线技术。
  - ✓ Thumb状态：bit[0]=0，bit[31:1]用于存放PC。





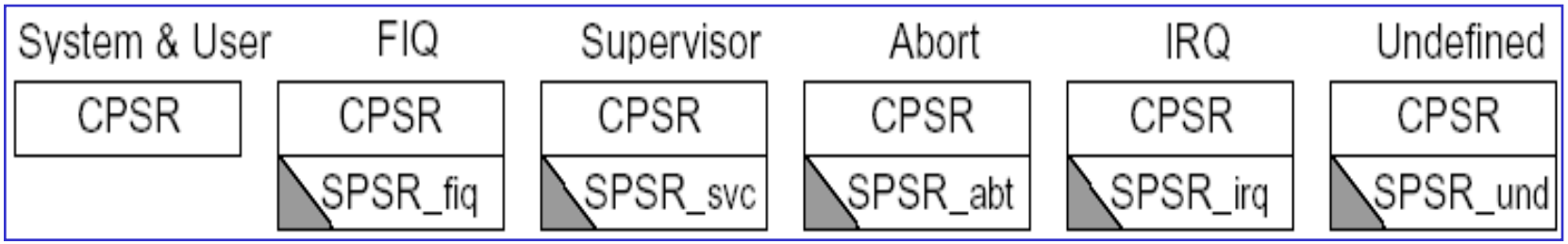


## ARM State General Registers and Program Counter

| System & User | FIQ | Supervisor | Abort | IRQ | Undefined |
|---|---|---|---|---|---|
| R0 | R0 | R0 | R0 | R0 | R0 |
| R1 | R1 | R1 | R1 | R1 | R1 |
| R2 | R2 | R2 | R2 | R2 | R2 |
| R3 | R3 | R3 | R3 | R3 | R3 |
| R4 | R4 | R4 | R4 | R4 | R4 |
| R5 | R5 | R5 | R5 | R5 | R5 |
| R6 | R6 | R6 | R6 | R6 | R6 |
| R7 | R7 | R7 | R7 | R7 | R7 |
| R8 | R8_fiq | R8 | R8 | R8 | R8 |
| R9 | R9_fiq | R9 | R9 | R9 | R9 |
| R10 | R10_fiq | R10 | R10 | R10 | R10 |
| R11 | R11_fiq | R11 | R11 | R11 | R11 |
| R12 | R12_fiq | R12 | R12 | R12 | R12 |
| R13 | R13_fiq | R13_svc | R13_abt | R13_irq | R13_und |
| R14 | R14_fiq | R14_svc | R14_abt | R14_irq | R14_und |
| R15 (PC) | R15 (PC) | R15 (PC) | R15 (PC) | R15 (PC) | R15 (PC) |

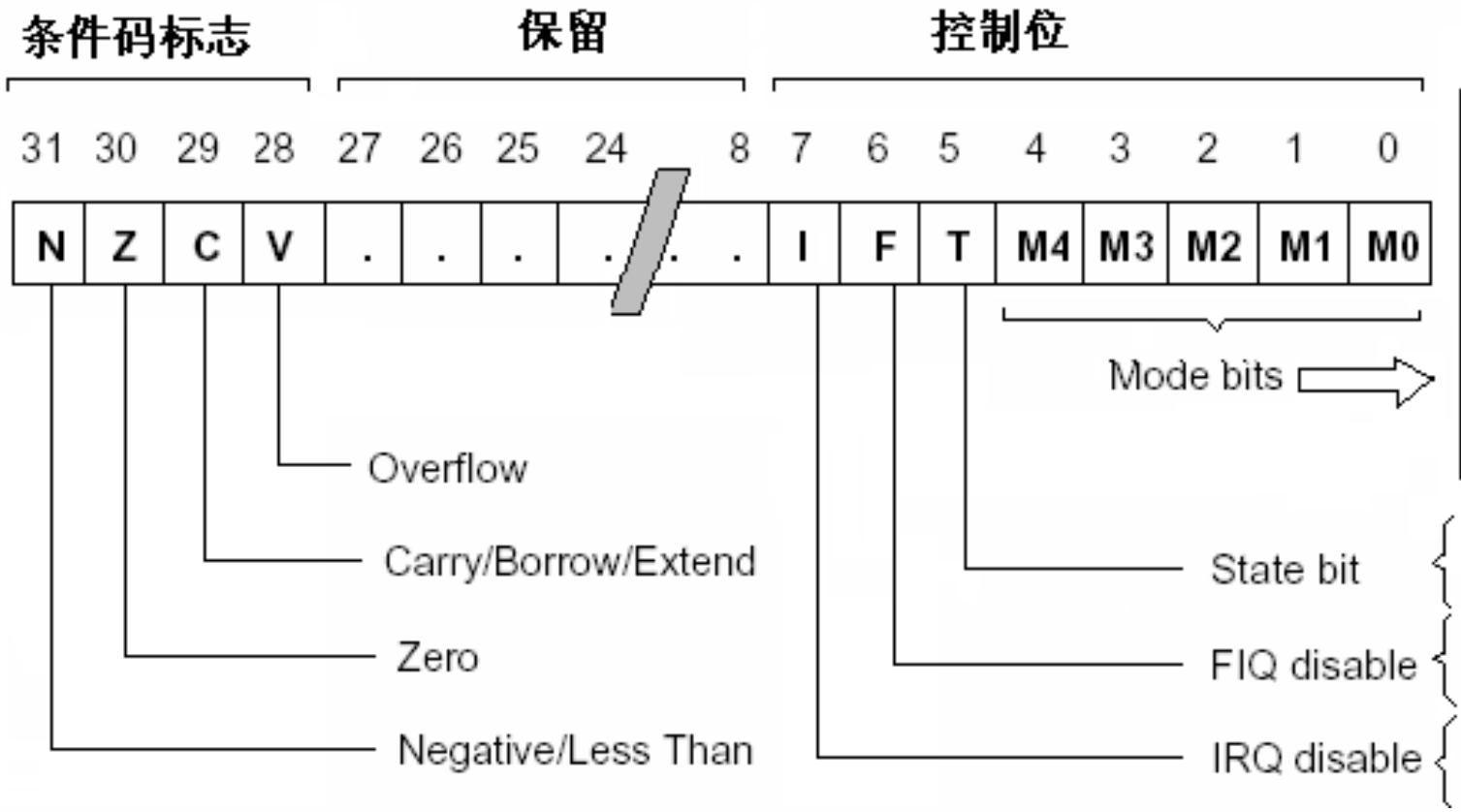
# 状态寄存器（6个）

- 当前程序状态寄存器CPSR (Current Program Status Register)
  - ✓ 所有运行模式下都指向相同的物理寄存器。可在任何运行模式下被访问。




- 备份程序状态寄存器SPSR (Saved Program Status Register)
  - ✓ 每种异常模式都有自己的专用物理寄存器。
  - ✓ 异常发生时，保存CPSR；异常退出时用于恢复CPSR。
  - ✓ 用户模式和系统模式下无效（访问时结果未知）。

# CPSR/SPSR寄存器格式：




| M[4: 0] | 模式 |
|---|---|
| 0b10000 | 用户 |
| 0b10001 | FIQ |
| 0b10010 | IRQ |
| 0b10011 | 管理 |
| 0b10111 | 中止 |
| 0b11011 | 未定义 |
| 0b11111 | 系统 |

## 条件码标志位

- N、Z、C、V标志位：它们的内容由算术或逻辑运算的结果决定，通常用于决定某条指令是否可以被执行。
  - ✓ ARM状态：绝大多数的指令都是有条件执行的。
  - ✓ Thumb状态：仅有分支指令（B指令）是有条件执行的。

## 保留位

- ✓ 为了提高程序的可移植性，当改变CPSR标志和控制位时，尽量不要改变保留位的值。
- ✓ 为了保证程序后续的兼容性，不要让保留位的值影响程序运行。

| 标志位 | 含　义 |
| --- | --- |
| N | 当用两个补码表示的带符号数进行运算时，N=1 表示运算的结果为负数；N=0 表示运算的结果为正数或零； |
| Z | Z=1 表示运算的结果为零；Z=0 表示运算的结果为非零； |
| C | 加法运算结果最高位进位时，C=1，否则C=0；<br>减法运算最高位借位时，C=0，否则C=1；<br>移位操作指令，C为从最高位最后移出的位；<br>其它指令，C的值通常不改变。 |
| V | 加/减法运算指令，若符号位溢出，则V=1，否则V=0；<br>其它指令，V的值通常不改变。 |
| Q | 在ARM v5及以上版本的E系列处理器中，Q标志指示DSP运算指令是否溢出。在其他版本中，Q标志位无定义。 |

# 控制位

- I、F、T和M[4:0]：发生异常时可以被*改变**。
- ★特别地：运行于特权模式时，可以由程序修改。
  - ✓ 中断禁止位I、F：决定中断能否发生。
    - I=1：禁止IRQ中断；
    - F=1：禁止FIQ中断。
  - ✓ 模式位M[4:0]：决定处理器的运行模式。
    - M[4:0]=10000：用户模式；
    - M[4:0]=10001：FIQ模式；
    - M[4:0]=10010：IRQ模式；
    - M[4:0]=10011：管理模式；
    - M[4:0]=10111：中止模式；
    - M[4:0]=11011：未定义；
    - M[4:0]=11111：系统模式。

注意：如果使用了错误的设置，将引起一个无法恢复的错误。

# 控制位（续）

- 工作状态位T：反映处理器的运行状态。
  - ARM体系结构v4及以上版本中T系列处理器
    - T=1：处理器运行于Thumb状态；
    - T=0：处理器运行于ARM状态。
  - ARM体系结构v4及以上版本中非T系列处理器
    - T=1：执行下一条指令并引起未定义指令异常；
    - T=0：处理器运行于ARM状态。

## Thumb状态

★Thumb状态下的寄存器组是ARM状态下寄存器组的一个子集。

- 通用寄存器
  - 21个
  - 32位
- 状态寄存器
  - 6个
  - 32位

ARM微处理器寄存器（27个）

(ARM7T、ARM9、ARM11系列)

## Thumb状态下的寄存器组

- 程序可以直接访问8个通用寄存器(R0～R7)、程序计数器(PC)、堆栈指针(SP)、链接寄存器(LR)。
- 有条件访问当前程序状态寄存器(CPSR)。
- 每一种特权模式下都有一组堆栈指针(SP)、链接寄存器(LR)和备份程序状态寄存器(SPSR)。

SP=R13，LR=R14，PC=R15







## THUMB State General Registers and Program Counter

| System & User | FIQ | Supervisor | Abort | IRQ | Undefined |
|---|---|---|---|---|---|
| R0 | R0 | R0 | R0 | R0 | R0 |
| R1 | R1 | R1 | R1 | R1 | R1 |
| R2 | R2 | R2 | R2 | R2 | R2 |
| R3 | R3 | R3 | R3 | R3 | R3 |
| R4 | R4 | R4 | R4 | R4 | R4 |
| R5 | R5 | R5 | R5 | R5 | R5 |
| R6 | R6 | R6 | R6 | R6 | R6 |
| R7 | R7 | R7 | R7 | R7 | R7 |
| SP | SP_fiq | SP_svc | SP_abt | SP_und | SP_fiq |
| LR | LR_fiq | LR_svc | LR_abt | LR_und | LR_fiq |
| PC | PC | PC | PC | PC | PC |

## THUMB State Program Status Registers

| | | | | | |
|---|---|---|---|---|---|
| CPSR | CPSR | CPSR | CPSR | CPSR | CPSR |
| | SPSR_fiq | SPSR_svc | SPSR_abt | SPSR_irq | SPSR_und |

# 两种状态下寄存器组的关系

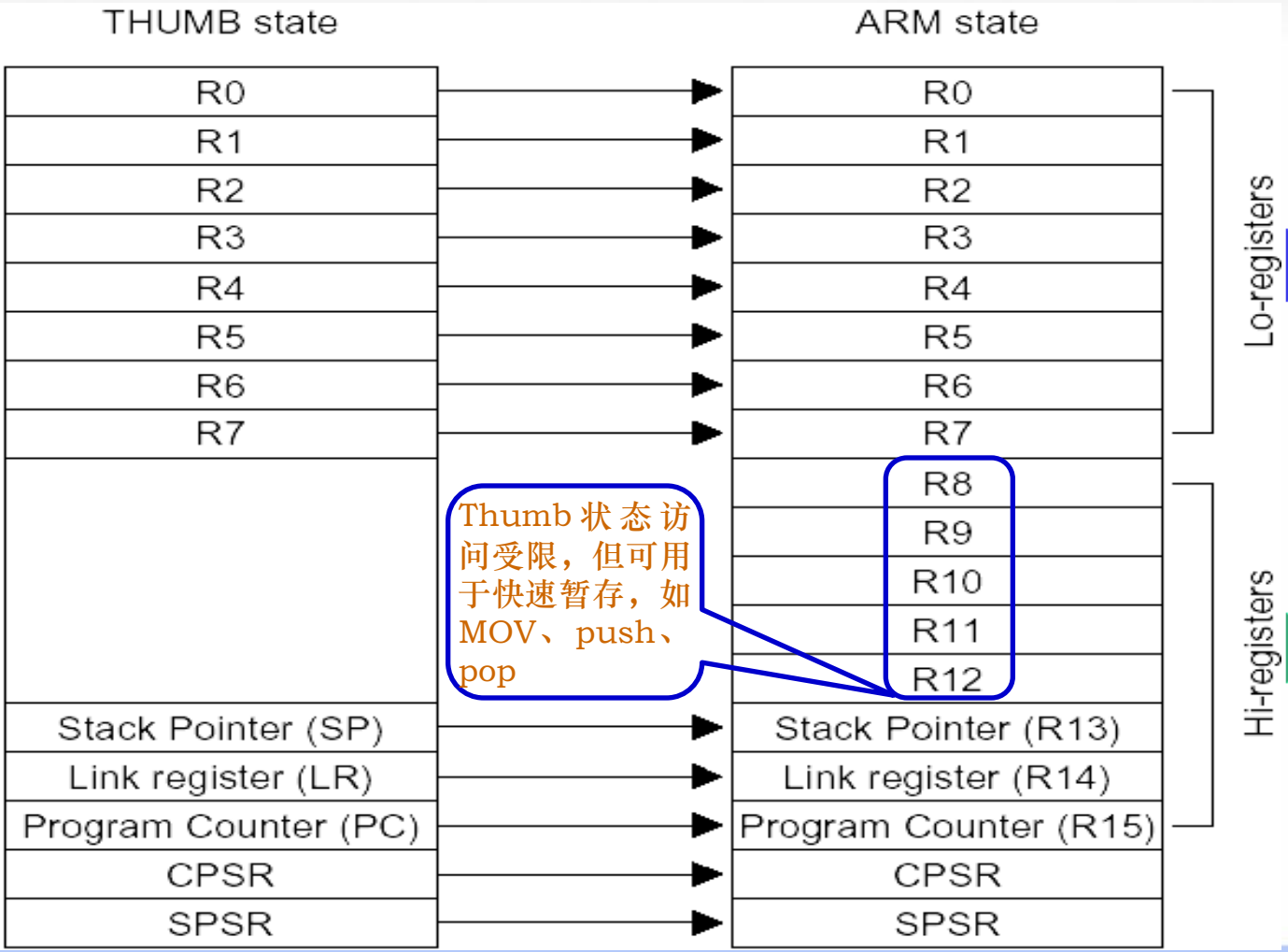
- 8个通用寄存器(R0～R7)完全相同。
  - ✓ 用于保存数据或地址值。
  - ✓ 对于任何运行模式，每一个都对应相同的32位物理寄存器。
  - ✓ 是完全通用的寄存器，不会被用作特殊用途，可用于任何使用通用寄存器的指令。
- CPSR和所有的SPSR完全相同。




-

| Thumb状态 | ARM状态 |
|---|---|
| SP | R13 |
| LR | R14 |
| PC | R15 |

## 两种状态下寄存器组的关系









- 2.1 微处理器相关概念
- 2.2 ARM微处理器概述
- 2.3 ARM微处理器结构
- 2.4 ARM微处理器运行模式
- 2.5 ARM微处理器工作状态
- 2.6 ARM微处理器寄存器组织
- **2.7 ARM微处理器存储器格式**
- 2.8 ARM微处理器异常处理

# ARM体系结构中的存储器格式

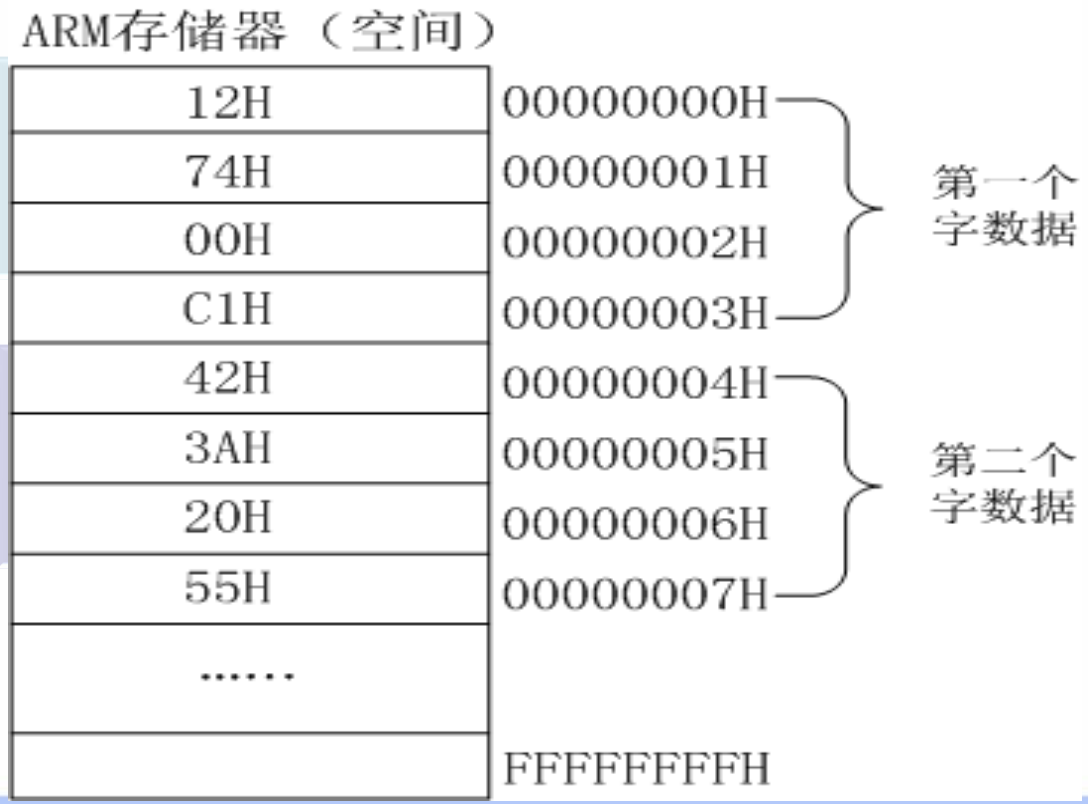
V8之前版本

从零地址开始的字节线性组合

寻址空间4GB

两种存储字数据的方法

- 从零字节到三字节放置第一个存储的字数据
- 从第四个字节到第七个字节放置第二个存储的字数据，依次排列



## 大端格式

- 字数据的高字节存储在低地址中，而字数据的低字节则存放在高地址中。

高地址



低地址

| 31 24 | 23 16 | 15 8 | 7 0 | 字地址 |
|---|---|---|---|---|
| 低字节 | | | 高字节 | 0x00000008 |
| 低字节 | | | 高字节 | 0x00000004 |
| 低字节 | | | 高字节 | 0x00000000 |

## 小端格式

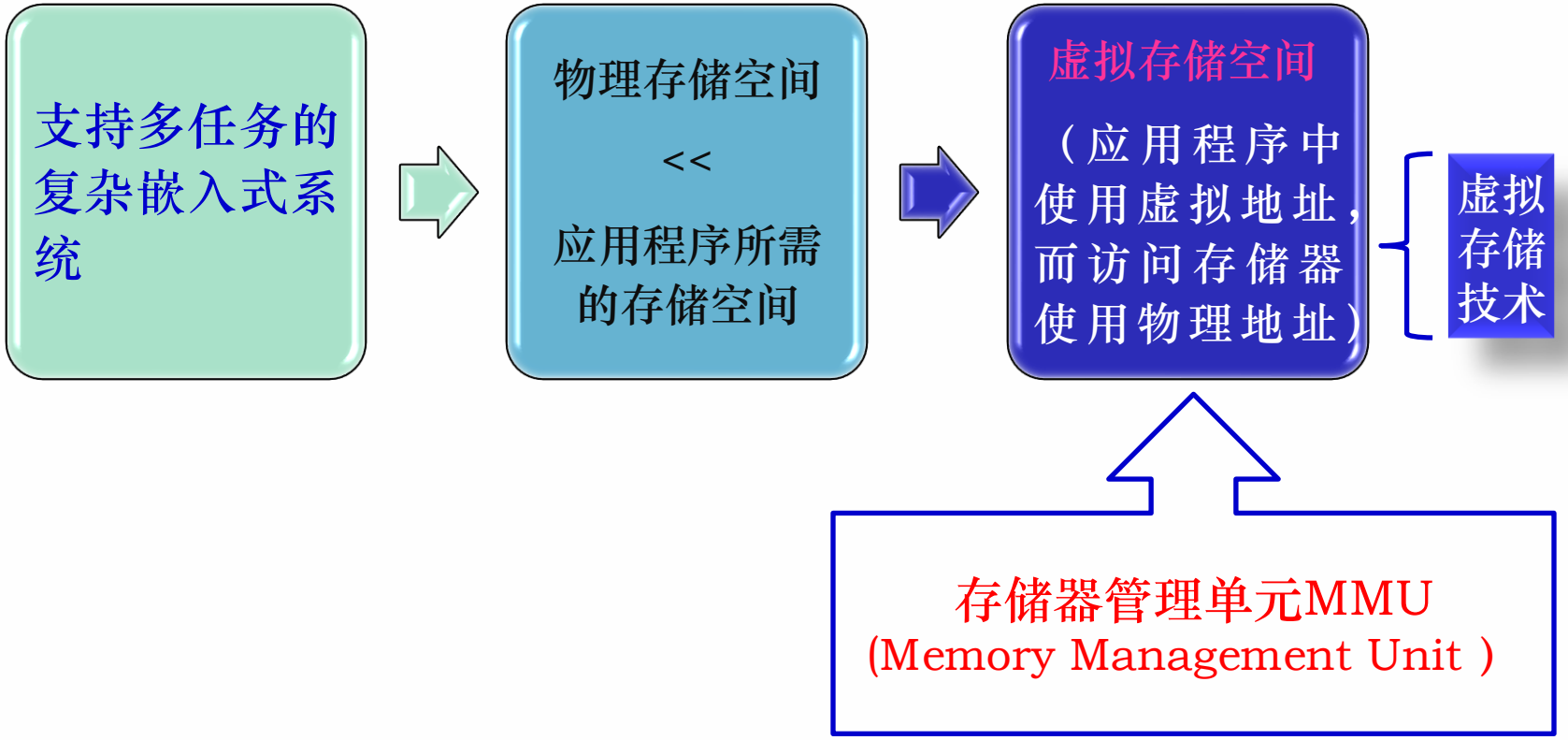
- 字数据的高字节存储在高地址中，而字数据的低字节则存放在低地址中。

高地址



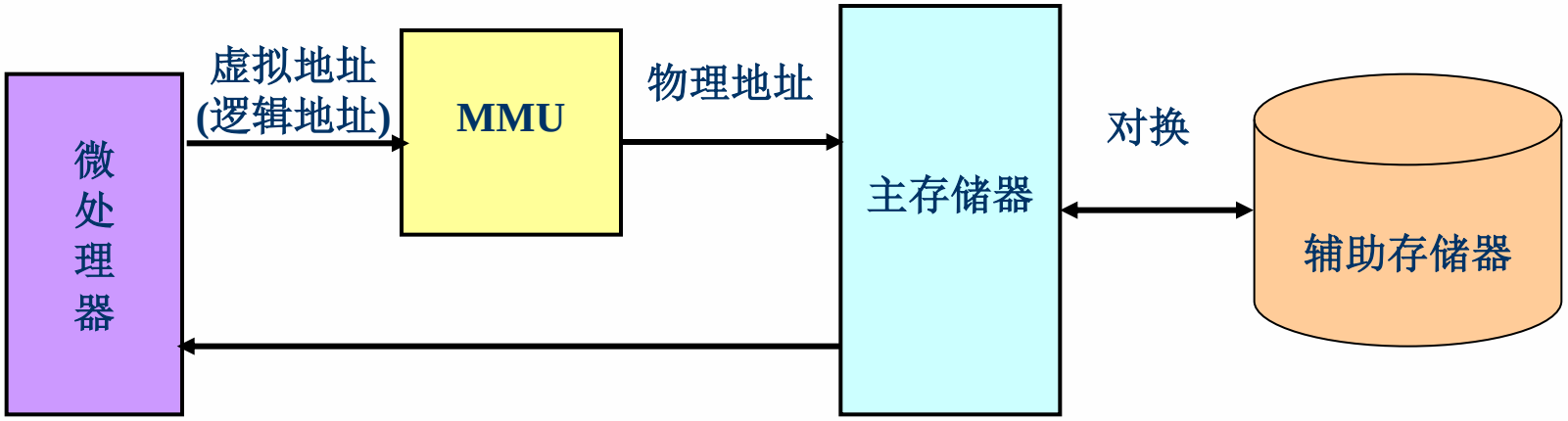
低地址

| 31 24 | 23 16 | 15 8 | 7 0 | 字地址 |
|---|---|---|---|---|
| 高字节 | | | 低字节 | 0x00000008 |
| 高字节 | | | 低字节 | 0x00000004 |
| 高字节 | | | 低字节 | 0x00000000 |


支持多任务的复杂嵌入式系统
物理存储空间
<<
应用程序所需的存储空间
虚拟存储空间
（应用程序中使用虚拟地址，而访问存储器使用物理地址）
虚拟存储技术
存储器管理单元MMU
(Memory Management Unit )

# 存储器管理单元MMU

- 将虚拟地址转换成物理地址——将主存地址从虚拟存储空间映射到物理存储空间。
- 存储器访问权限控制。
- 设置虚拟存储空间的缓冲特性等。



## 虚拟存储管理

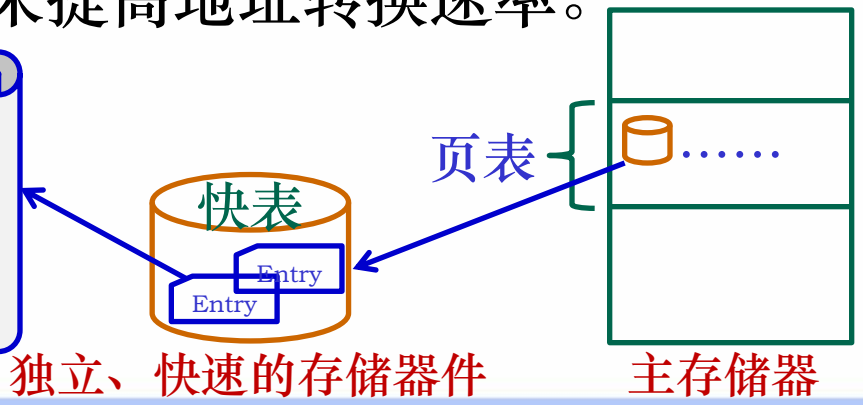
- 分段式存储管理
- **分页式存储管理**
- 段页式存储管理

## 分页式存储管理

ARM920T微处理器

- 把虚拟存储空间分成一个个固定大小的页(Page)，把物理主存储的空间也分成同样大小*的一个个页框(Frame)。
- 通过查询存放在主存中的页表(Page Table)，来实现虚拟地址到物理地址的转换。——对照表
- 由于查询页表代价很大，故采用快表技术(Translation Lookaside Buffer, TLB)来提高地址转换速率。

地址转换条目(Entry)：表示虚拟地址和物理地址对应关系的记录，也称为页表条目、页表项、表项。

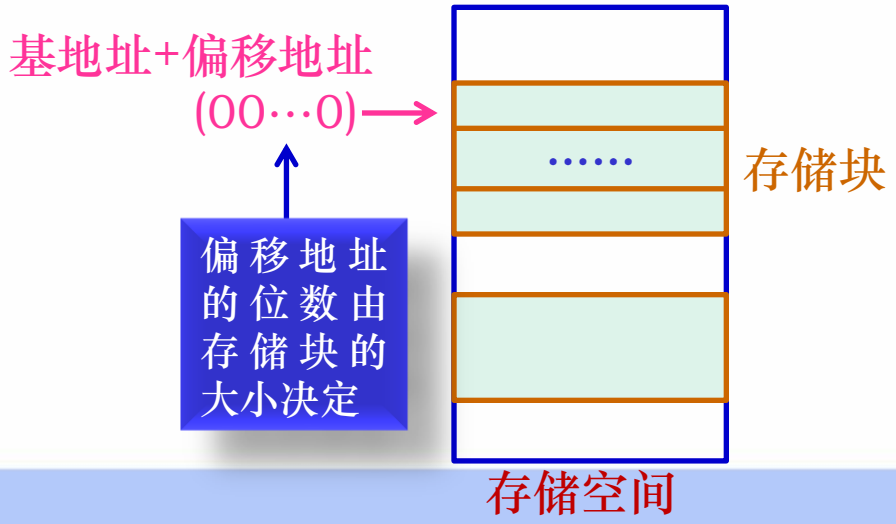


## 基于TLB的地址转换过程

- 微处理器访问主存时，首先根据虚拟地址在TLB中查找对应的转换条目。如果存在(命中，Hit)，则直接转换。
- 如果不存在，则继续在位于主存的页表中查找，并把查找到结果(地址转换条目)添加到TLB中。——TLB更新
- 在添加时，如果TLB已满，可根据一定的淘汰算法进行替换。

## 两种存储空间的映射单位

- 虚拟存储空间到物理存储空间的映射是以存储块(Memory Block)为单位进行的。
- 在页表/TLB中，每个Entry记录了一个虚拟存储空间的存储块的基地址与物理存储空间的一个存储块的基地址的对应关系。

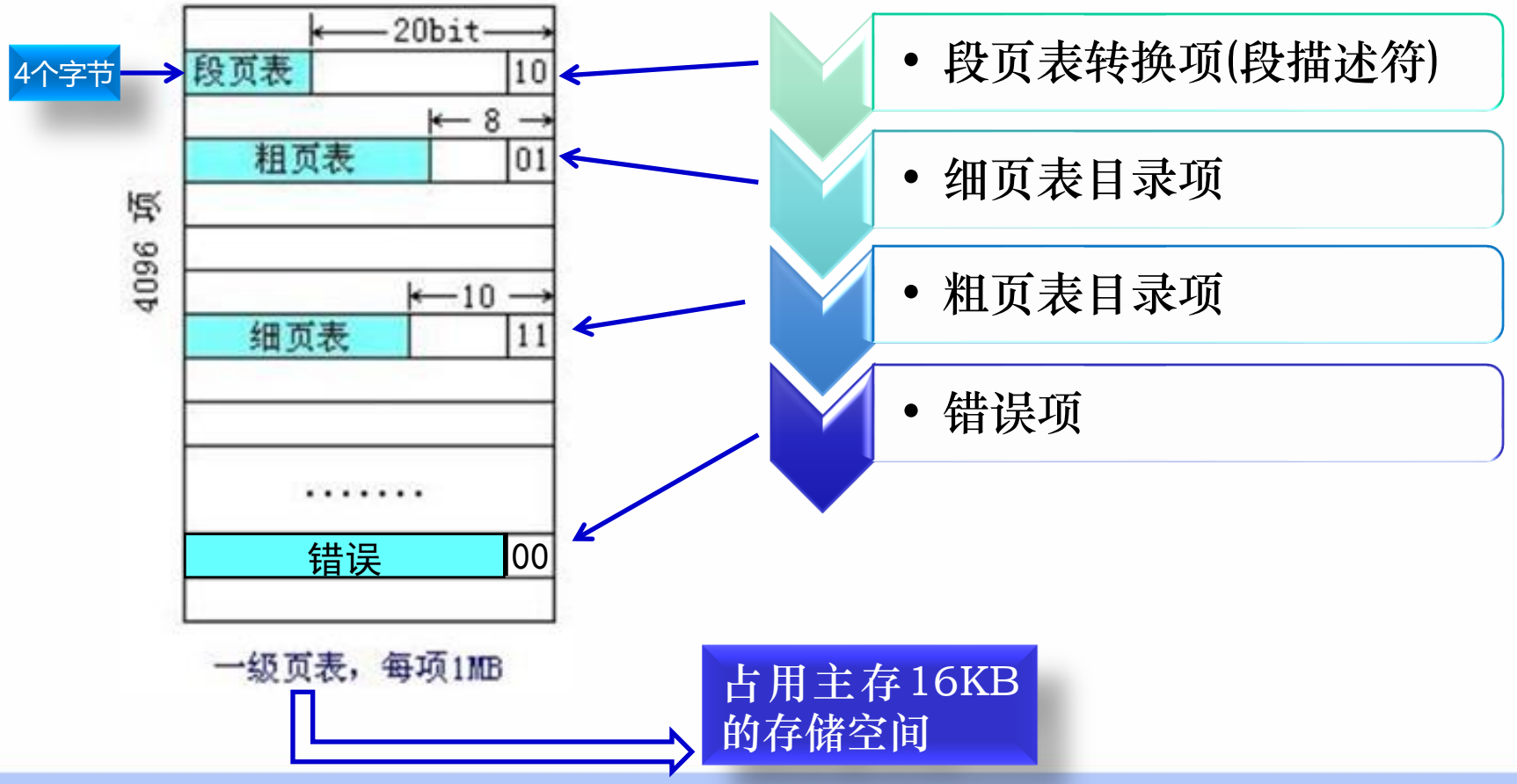


## ARM920T支持的存储块类型

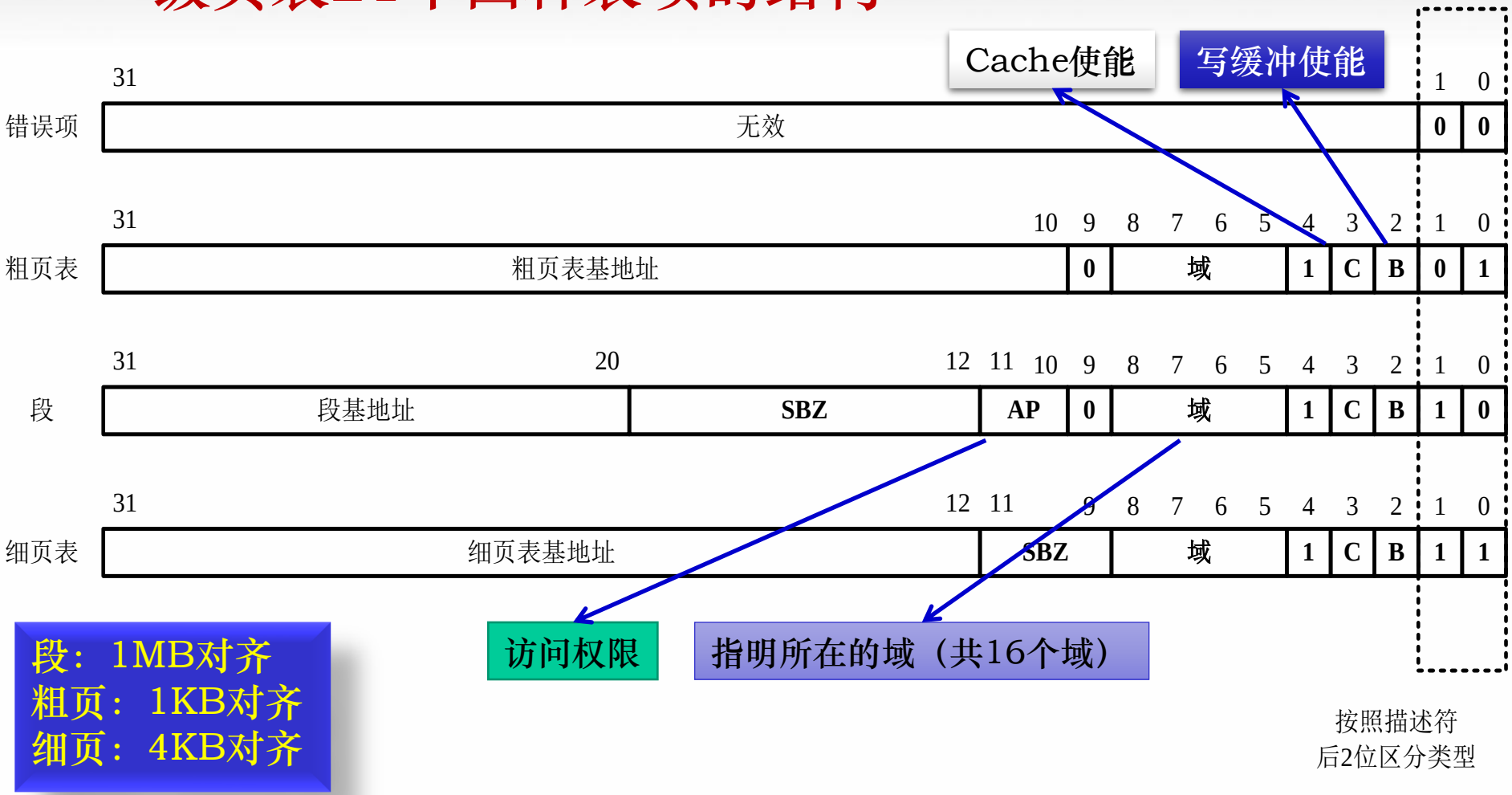
地址转换条目

| | | |
|---|---|---|
| 段描述符 | 段 (section) | • 大小为1MB的存储块。 |
| 大页描述符 | 大页 (large pages) | • 大小为64KB的存储块。 |
| 小页描述符 | 小页 (small pages) | • 大小为4KB的存储块。 |
| 极小页描述符 | 极小页 (tiny pages) | • 大小为1KB的存储块。 |

## 两级页表结构：一级页表L1 + 二级页表L2



# 一级页表L1中四种表项的结构



## 两级页表结构：一级页表L1 + 二级页表L2

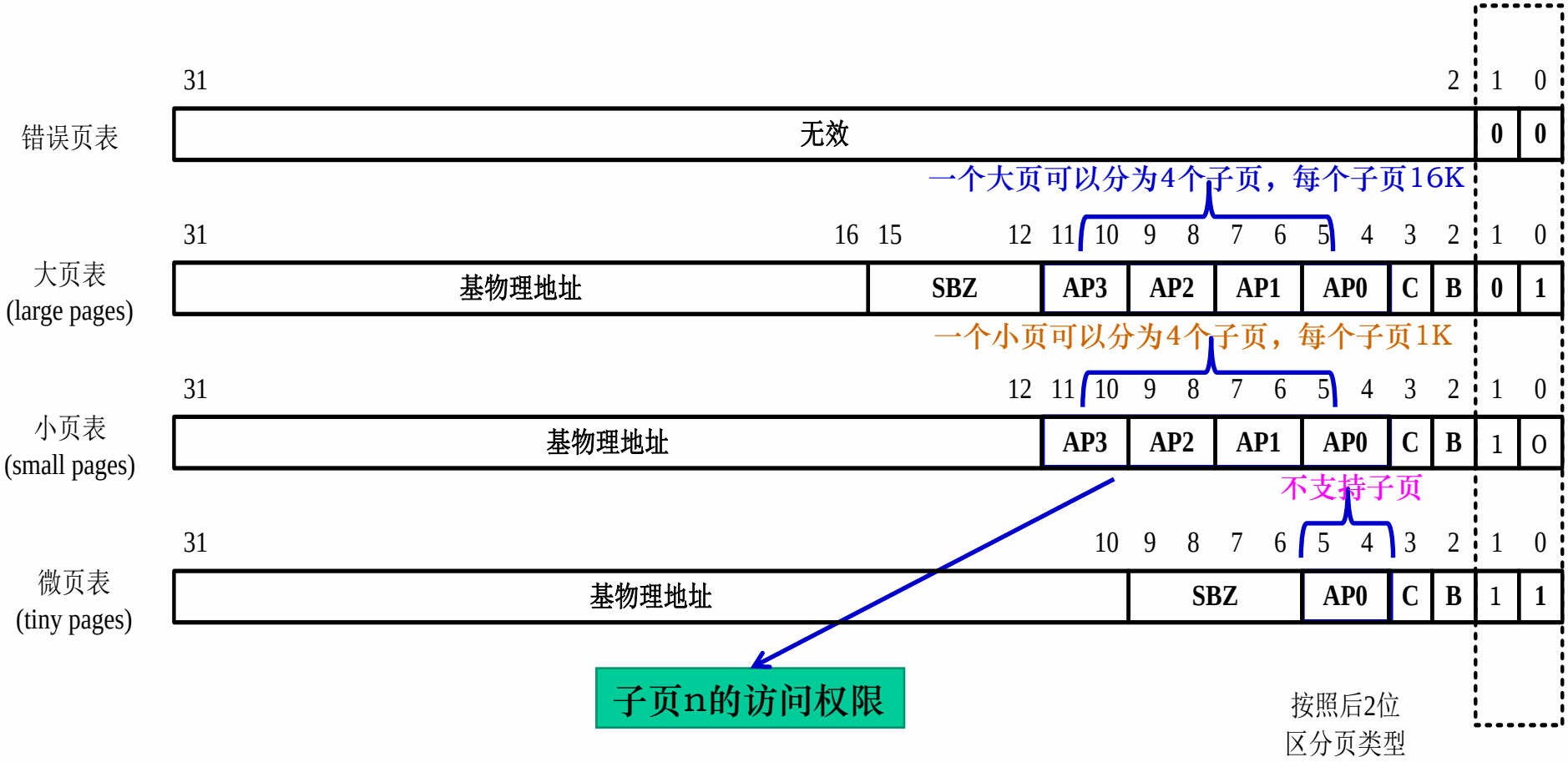



占用主存1KB的存储空间

- 大页表转换项(大页描述符)
- 小页表转换项(小页描述符)
- 极小页表转换项(极小页描述符)
- 页错误项

占用主存4KB的存储空间

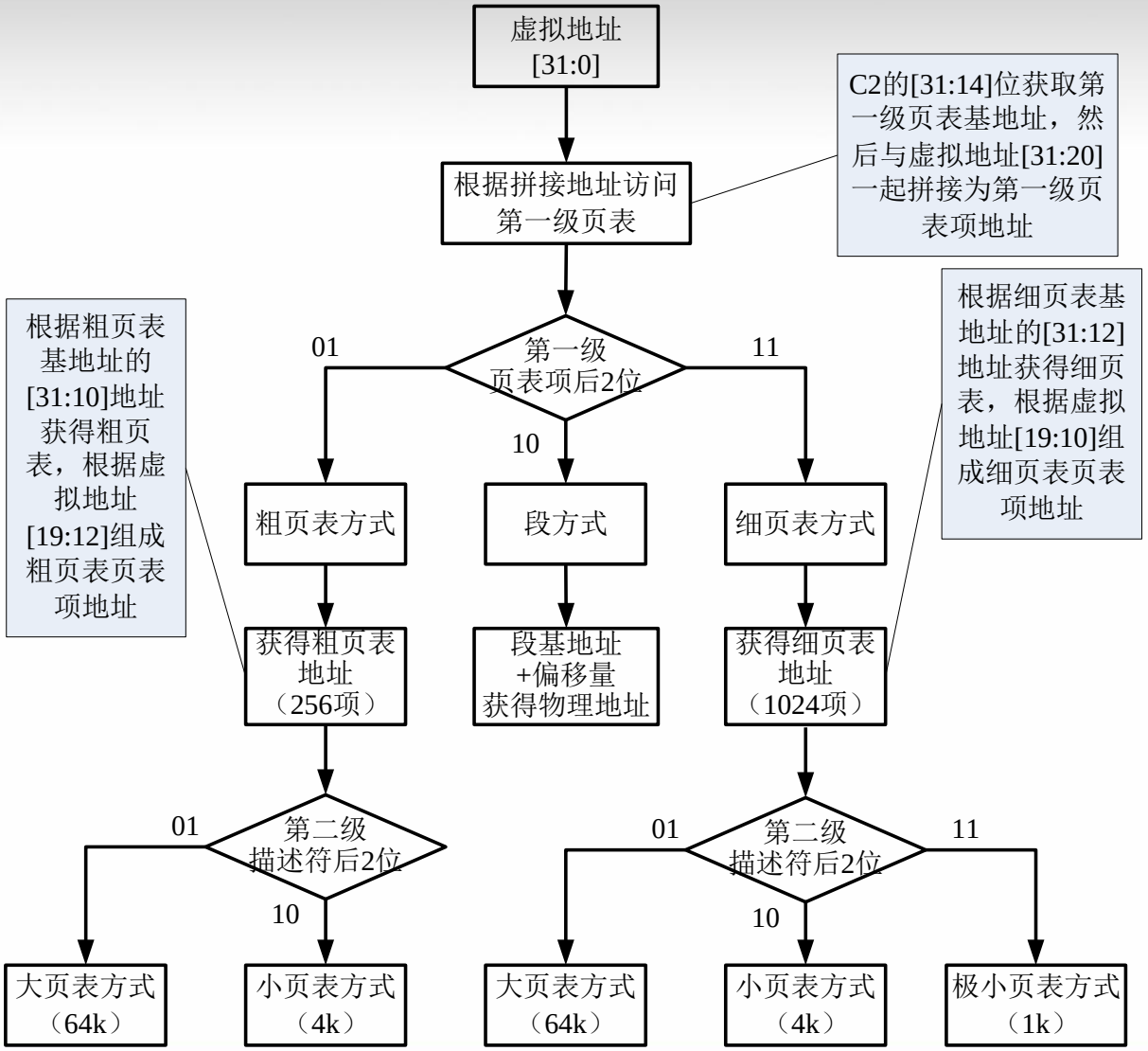
# 二级页表L2中四种表项的结构



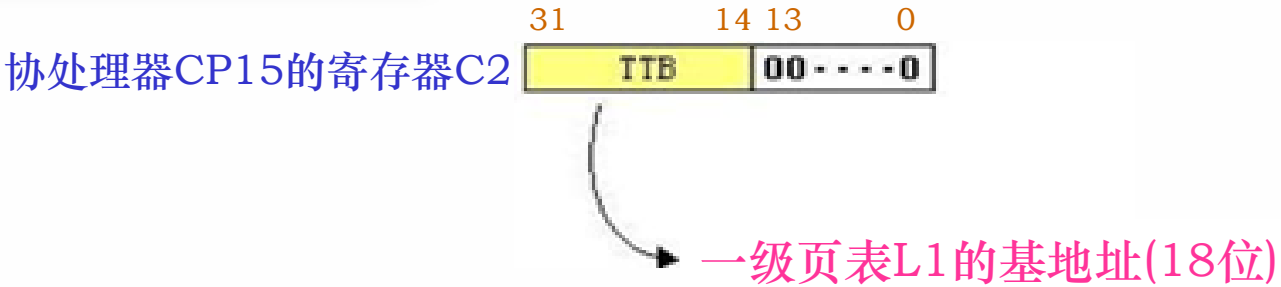
| 页表类型 | 页表项数 | 页表占用内存 | 支持页的单位 |
| --- | --- | --- | --- |
| 一级页表 | 4096 | 16k | 1M |
| 粗页表 (Coarse Page) | 256 | 1k | 4k或者64k |
| 细页表 (Fine Page) | 1024 | 4k | 1k、4k或者64k |

## 地 址 转 换 过 程

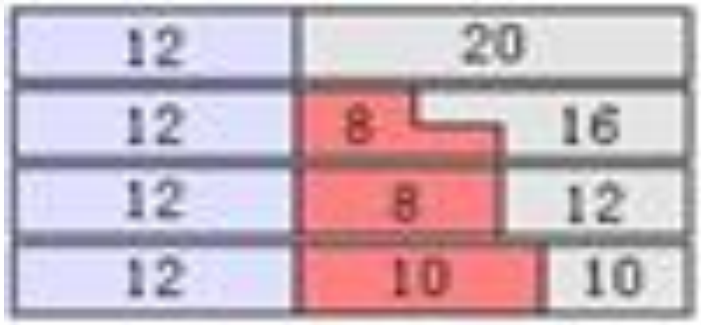
- (1) 根据给定的虚拟地址查找一级页表，获得一个条目。
- (2) 若该条目是段描述符，则返回物理地址，转换结束。——单步搜索
- (3) 若该条目是粗页表目录项，则继续利用虚拟地址查找二级粗页表，获得大页描述符或小页描述符，返回物理地址，转换结束。——两步搜索
- (4) 若该条目是细页表目录项，则继续利用虚拟地址查找二级细页表，获得大页、小页或极小页描述符，返回物理地址，转换结束。——两步搜索
- (5) 其它情况出错。


虚拟地址
[31:0]
根据拼接地址访问
第一级页表
C2的[31:14]位获取第一级页表基地址，然后与虚拟地址[31:20]一起拼接为第一级页表项地址
第一级
页表项后2位
01
10
11
根据粗页表基地址的[31:10]地址获得粗页表，根据虚拟地址[19:12]组成粗页表页表项地址
根据细页表基地址的[31:12]地址获得细页表，根据虚拟地址[19:10]组成细页表页表项地址
粗页表方式
段方式
细页表方式
获得粗页表地址
（256项）
段基地址
+偏移量
获得物理地址
获得细页表地址
（1024项）
第二级
描述符后2位
01
10
11
大页表方式
（64k）
小页表方式
（4k）
大页表方式
（64k）
小页表方式
（4k）
极小页表方式
（1k）

## 一级页表L1的基地址

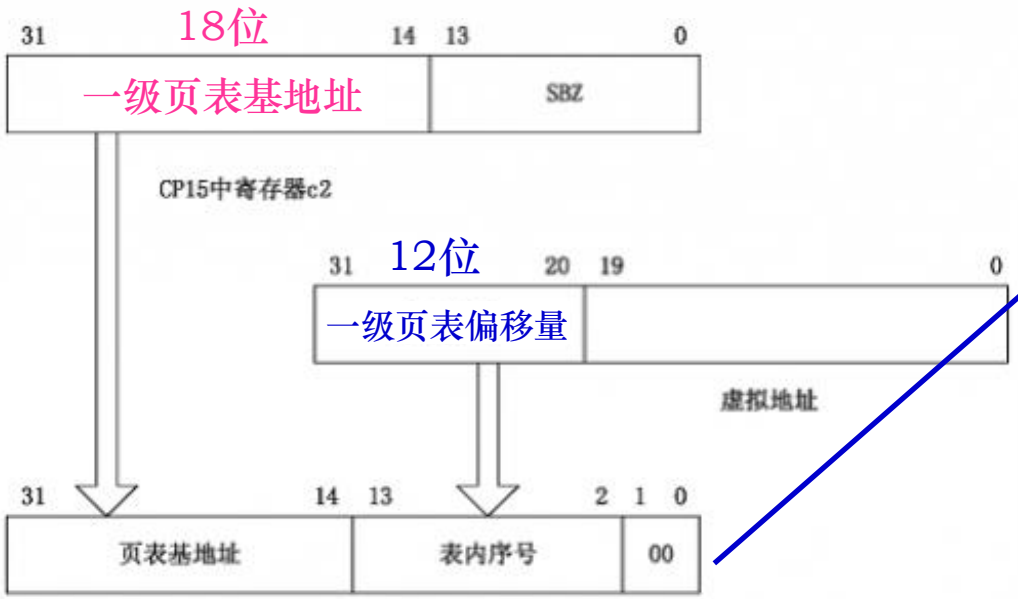



## 虚拟地址的结构




段 (1M)
大页 (64K)
小页 (4K)
极小页 (1K)

# 定位一级页表的过程

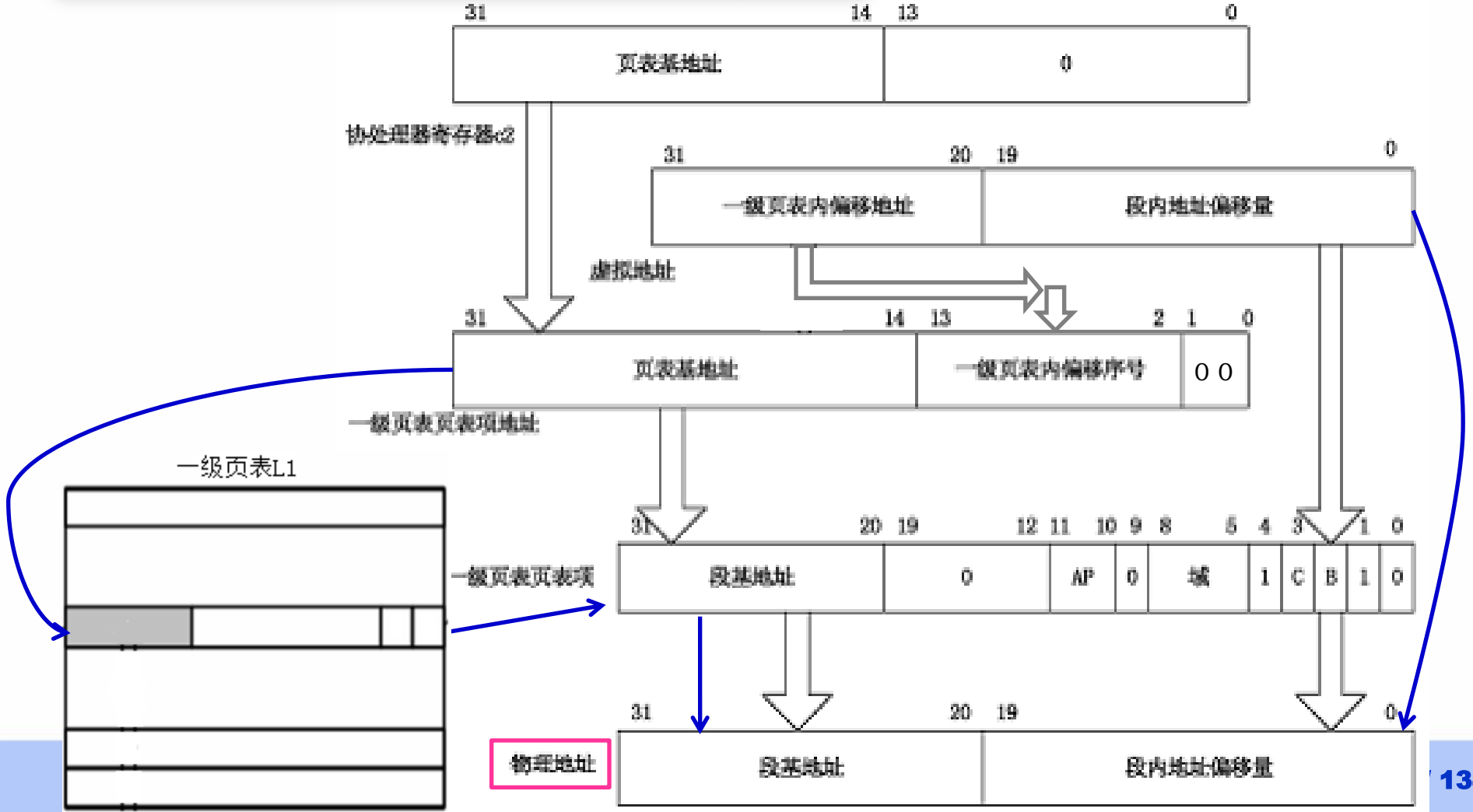



合成一级页表中某个表项的地址



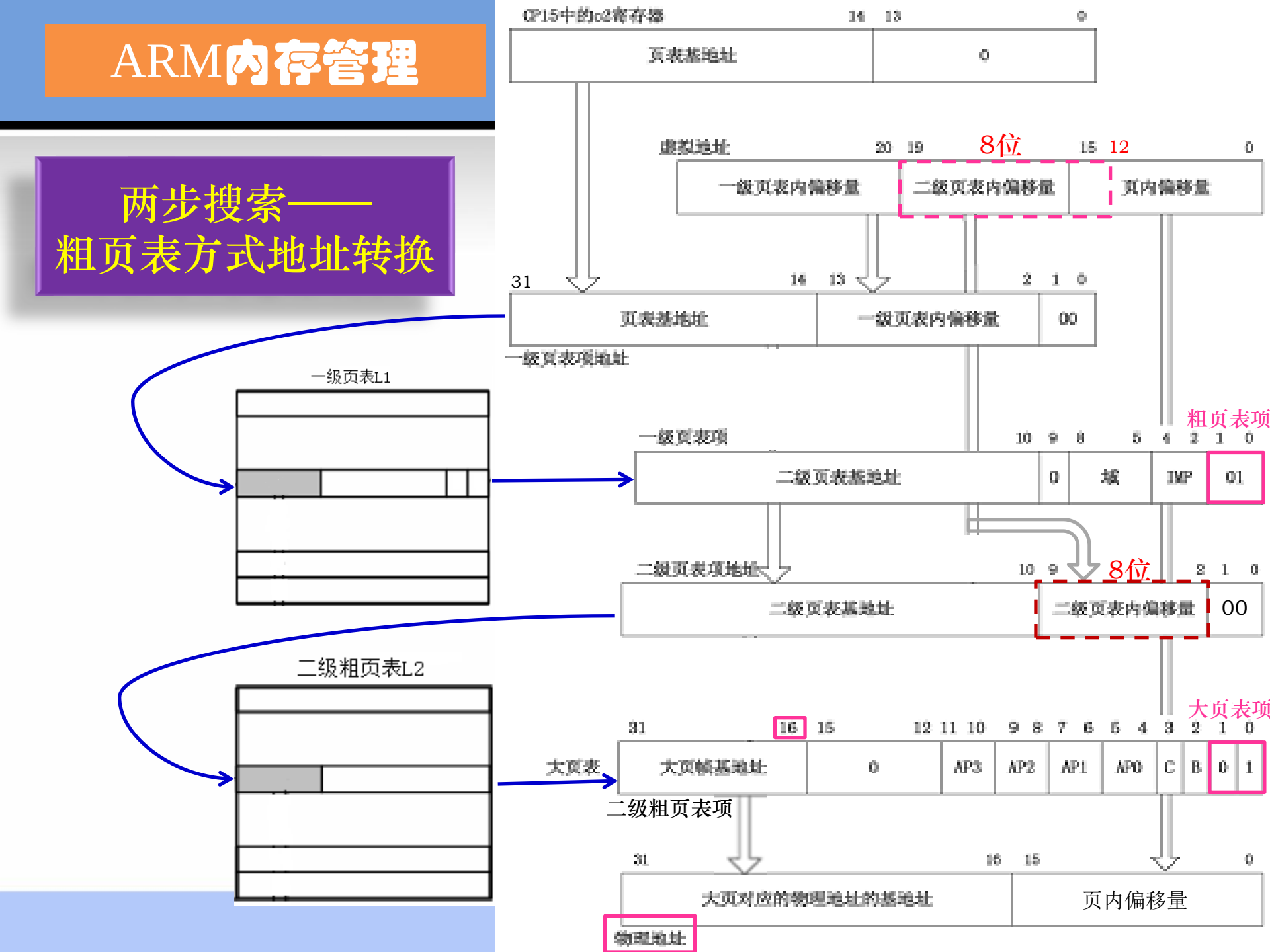

一级页表，每项1MB

# 单步搜索——段方式地址转换



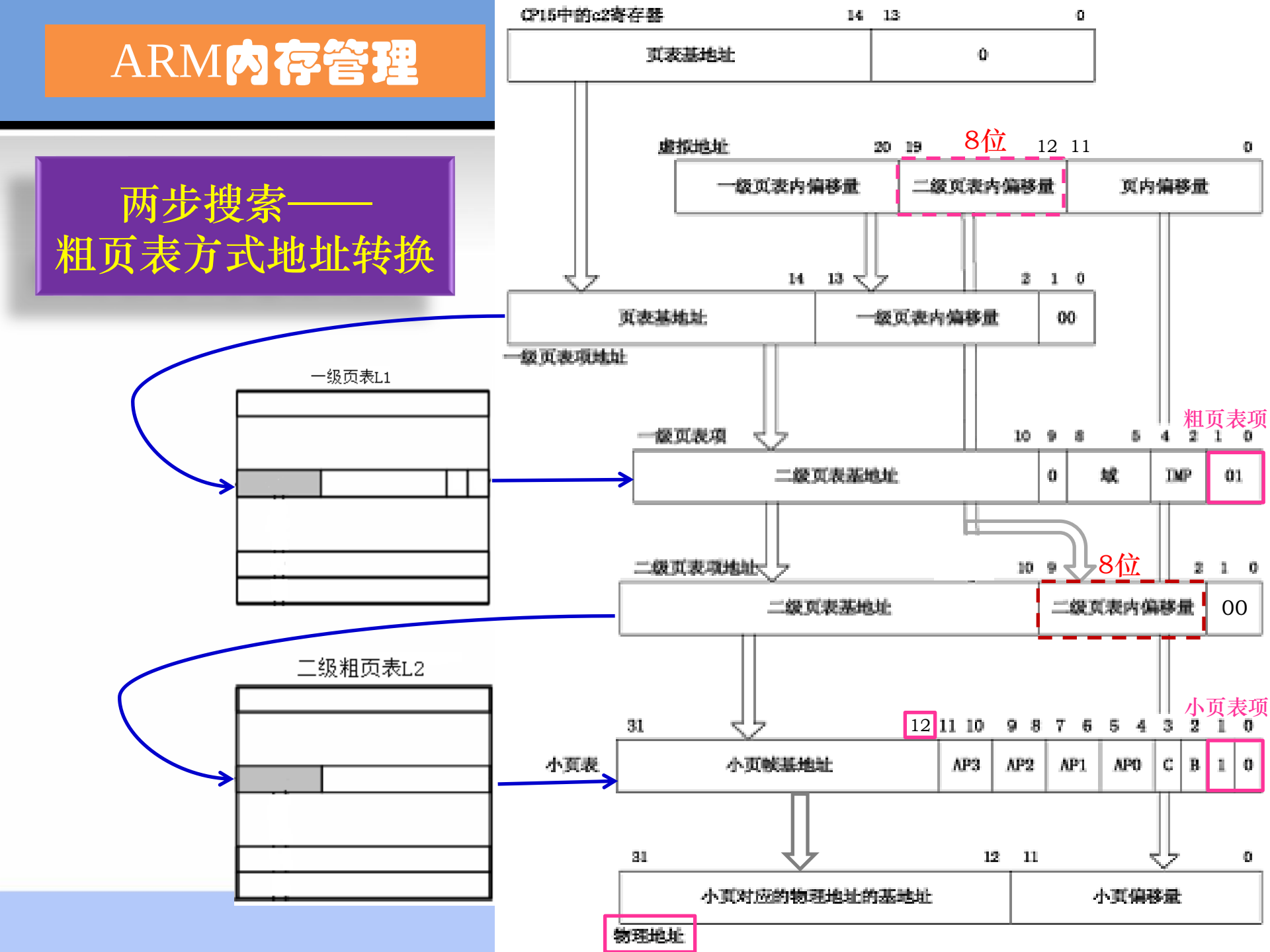
# ARM内存管理

## 两步搜索——粗页表方式地址转换



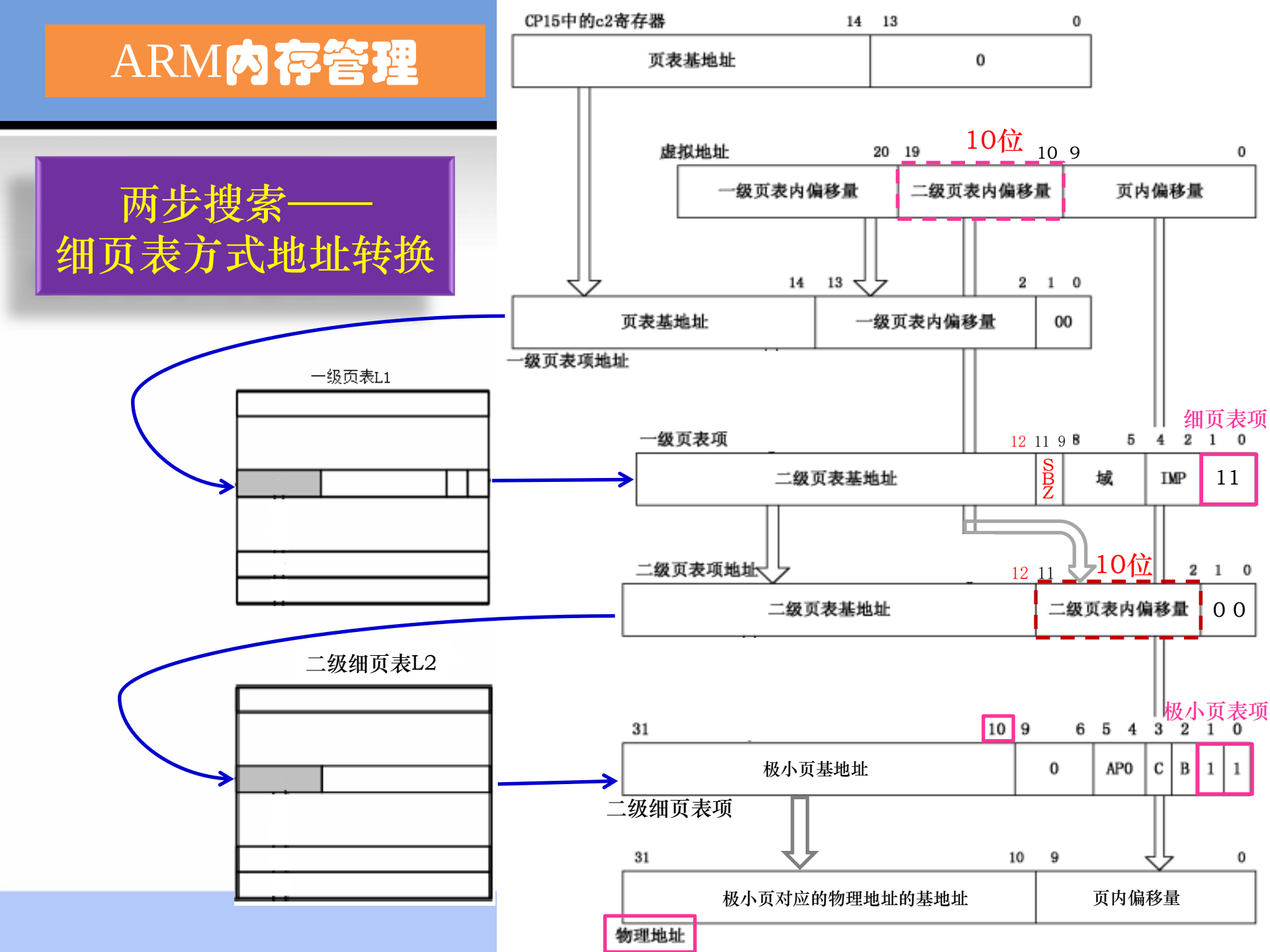
# ARM内存管理

## 两步搜索——粗页表方式地址转换



# ARM内存管理

## 两步搜索——细页表方式地址转换



# CP15——系统控制协处理器（System Control Coprocessor）

- 包含16个32位的寄存器，编号为0～15。
- 对MMU进行配置，完成存储器系统管理。
  - ✓ 页表和TLB。
  - ✓ 域和访问权限。
  - ✓ Cache和写缓冲器(Write Buffer)。
  - ✓ 快速上下文切换。

# CP15中与MMU相关的寄存器

| 寄　存　器 | 作　　用 |
| --- | --- |
| C1中某些位 | 用于配置MMU中的一些操作 |
| C2 | 保存主存中页表的基地址 |
| C3 | 设置域（domain）的访问控制属性 |
| C4 | 保留 |
| C5 | 主存访问失效状态指示 |
| C6 | 主存访问失效时的地址 |
| C8 | 控制与清除TLB内容相关的操作 |
| C10 | 控制与锁定TLB内容相关的操作 |

# 禁止与使能MMU

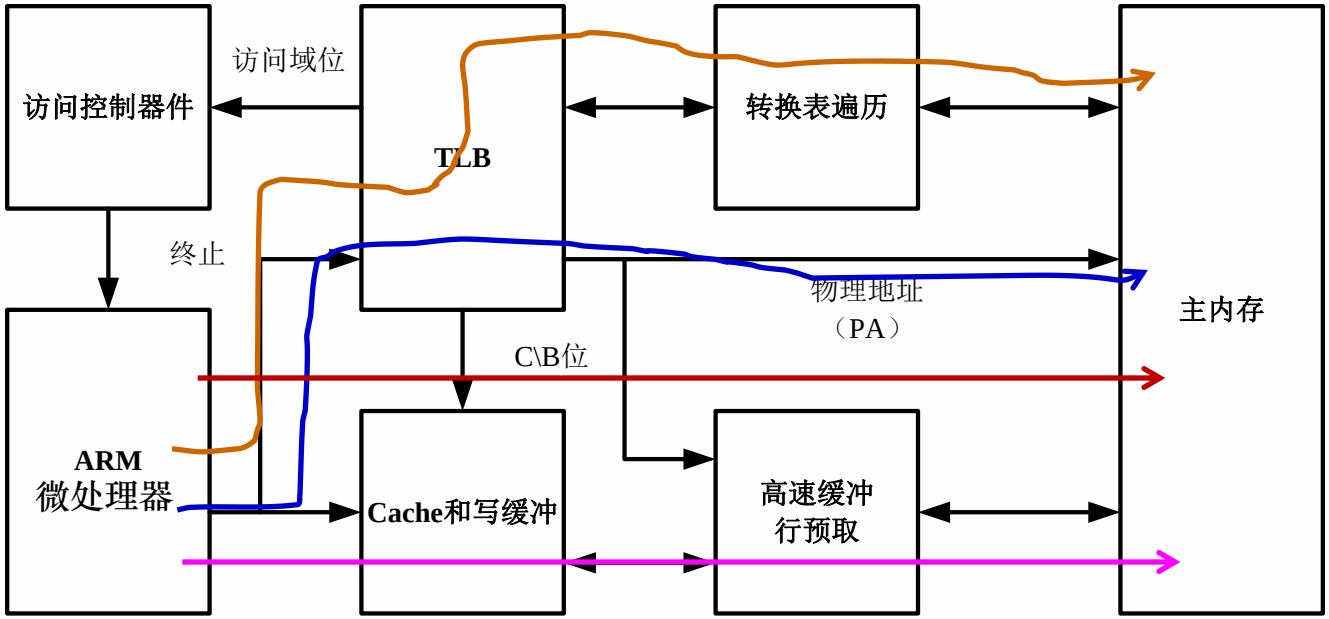
- 寄存器C1的位[0]用于设置禁止/使能MMU。
  - ✓ C1的位[0]=0时，禁止MMU；
  - ✓ C1的位[0]=1时，使能MMU。

实现使能MMU的汇编指令

```
MRC    P15,0,R0,C1,C0,0      ;C1的内容赋给R0
ORR    R0, R0,#0x1
MCR    P15,0,R0,C1,C0,0      ;R0的内容赋给C1
```

# 禁止MMU

- 虚拟地址=物理地址——平板地址映射(Flat Address Mapping)模式。
- 没有存储访问权限控制，MMU也不会产生异常信号。
- 是否支持Cache和写缓冲由具体芯片设计确定。



## 禁止/使能MMU的注意事项

- 使能MMU之前，要在主存中建立页表，同时CP15相关寄存器必须完成初始化。
- 若设计的物理地址空间与虚拟地址空间不相等，则在禁止/使能MMU时，虚拟地址和物理地址的对应关系会改变，故应清除Cache中当前地址转换条目。
- 完成禁止/使能MMU程序，其物理地址最好与虚拟地址相同，否则禁止/使能MMU将带来很大麻烦。

## 存储访问权限控制

- CP15中C1的S和R控制位 & 页表中地址转换条目的访问权限控制位 → 控制存储访问权限。

| AP | S | R | 特权级时访问权限 | 用户级时访问权限 |
|---|---|---|---|---|
| 0b00 | 0 | 0 | 没有访问权限 | 没有访问权限 |
| 0b00 | 1 | 0 | 只读 | 没有访问权限 |
| 0b00 | 0 | 1 | 只读 | 只读 |
| 0b00 | 1 | 1 | 不可预知 | 不可预知 |
| 0b01 | X | X | 读/写 | 没有访问权限 |
| 0b10 | X | X | 读/写 | 只读 |
| 0b11 | X | X | 读/写 | 读/写 |

## 域

- 定义：一些段、大页或者小页的集合。
- ARM920T支持最大16个域，每个域的访问控制特性由CP15中C3的两位来控制。——每两位控制一个域

| 控制位编码 | 访问类型 | 含　义 |
| --- | --- | --- |
| 0b00 | 没有访问权限 | 若访问该域将产生访问失效 |
| 0b01 | 客户类型 | 根据页表中地址转换条目的访问权限控制位，决定是否允许特定的存储访问 |
| 0b10 | 保留 | 使用该值会产生不可预知的结果 |
| 0b11 | 管理者权限 | 不考虑页表中地址转换条目的访问权限控制位，这种情况下不会产生访问失效 |

# 存储访问失效

ARM920T微处理器

- MMU可以产生4种类型的存储访问失效。

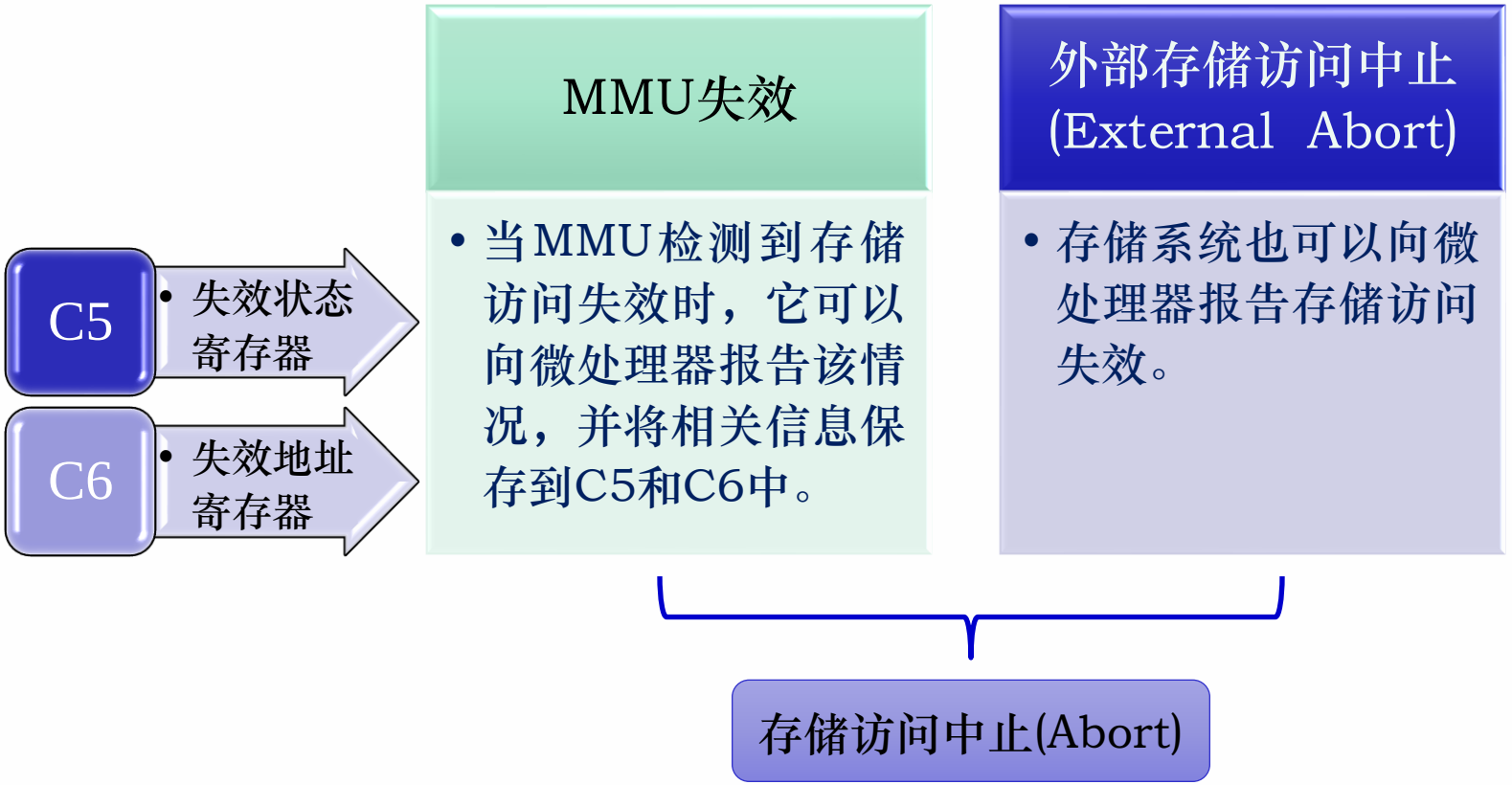
1. 地址对齐失效
2. 地址变换失效
3. 域控制失效
4. 访问权限控制失效

- 发生存储访问失效时，存储系统可以中止3种存储访问。

Cache内容预取　非缓冲的存储器访问操作　页表访问

# 存储访问失效

- 检测机制。


MMU失效
• 当MMU检测到存储访问失效时，它可以向微处理器报告该情况，并将相关信息保存到C5和C6中。
外部存储访问中止(External Abort)
• 存储系统也可以向微处理器报告存储访问失效。
C5
• 失效状态寄存器
C6
• 失效地址寄存器
存储访问中止(Abort)

# 存储访问失效

- 存储访问中止异常。

| 数据访问中止异常 | 指令预取异常 |
| --- | --- |
| • 在数据访问周期出现存储访问失效时产生。 | • 在指令预取周期出现存储访问失效，在执行该指令时产生。 |







- 2.1 微处理器相关概念
- 2.2 ARM微处理器概述
- 2.3 ARM微处理器结构
- 2.4 ARM微处理器运行模式
- 2.5 ARM微处理器工作状态
- 2.6 ARM微处理器寄存器组织
- 2.7 ARM微处理器存储器格式
- **2.8 ARM微处理器异常处理**

## ARM控制程序执行流程的方式

- 正常执行：每执行一条ARM指令，程序计数器寄存器(PC)的值加4个字节；每执行一条Thumb指令，PC的值加2个字节。整个过程按顺序执行。
- 通过跳转指令(B、BL、BLX和BX)，程序可以跳转到特定的位置处(地址标号、子程序等）执行。
- 正常的程序执行流程发生暂时的停止，称之为异常。例如：处理一个外部的中断请求。







## 异常处理的过程

| 异常类型 | 优先级 |
| --- | --- |
| 复位 | 1（最高优先级） |
| 数据中止 | 2 |
| FIQ | 3 |
| IRQ | 4 |
| 预取指令中止 | 5 |
| 未定义指令 | 6 |
| SWI | 7（最低优先级） |

优先级降低

统执行
场)，然
的执行
。
的优先

## 异常处理的过程

- 当异常发生时(例如处理一个外部的中断请求)，系统执行完当前指令后，需要保存处理器的当前状态(执行现场)，然后将跳转到相应的异常处理程序处执行。
- 当异常处理程序执行完成后，程序应恢复所保存的执行现场，从而返回到发生异常时的下一条指令继续执行。
- 处理器允许多个异常同时发生，它们将会按固定的优先级进行处理。

| 异常类型 | 进入模式 | 具体含义 | 异常向量地址 |
|---|---|---|---|
| 复位 | 管理 | 复位电平有效时，产生复位异常，程序跳转到复位处理程序处执行。 | 0x00000000 |
| 未定义指令 | 未定义 | 遇到不能处理的指令时，产生未定义指令异常。 | 0x00000004 |
| 软件中断（SWI） | 管理 | 执行SWI指令产生，用于用户模式下的程序调用特权操作指令。 | 0x00000008 |
| 指令预取中止 | 中止 | 处理器预取指令的地址不存在，或该地址不允许当前指令访问，产生指令预取中止异常。 | 0x0000000C |
| 数据访问中止 | 中止 | 处理器数据访问指令的地址不存在，或该地址不允许当前指令访问时，产生数据中止异常。 | 0x00000010 |
| IRQ（中断） | IRQ | 外部中断请求有效，且CPSR中的I位为0时产生IRQ异常。 | 0x00000018 |
| FIQ（快速中断） | FIQ | 快速中断请求有效，且CPSR中的F位为0时产生FIQ异常。 | 0x0000001C |

# 关于异常向量的几个概念

- 异常向量地址(Exception Vectors)：异常出现后，强制从异常类型对应的固定地址开始执行程序。
- 异常向量表：由七个异常向量及其异常处理函数跳转关系组成的表。每个异常向量占4个字节。(★表现为一条跳转指令)

```
0x00000000: b reset
0x00000004: ldr pc, _undefined_instruction
0x00000008: ldr pc, _software_interrupt
0x0000000c: ldr pc, _prefetch_abort
0x00000010: ldr pc, _data_abort
0x00000014: ldr pc, _not_used
0x00000018: ldr pc, _irq
0x0000001c: ldr pc, _fiq
```









| 异常类型 | 模式 | 低向量地址<br>(默认使用) | 高向量地址 |
|---|---|---|---|
| 复位 | 管理 | 0x00000000 | 0xFFFF0000 |
| 未定义指令 | 未定义 | 0x00000004 | 0xFFFF0004 |
| 软件中断（SWI） | 管理 | 0x00000008 | 0xFFFF0008 |
| 预取中止<br>（取指令存储器中止） | 中止 | 0x0000000C | 0xFFFF000C |
| 数据中止<br>（数据访问存储器中止） | 中止 | 0x00000010 | 0xFFFF0010 |
| IRQ（中断） | IRQ | 0x00000018 | 0xFFFF0018 |
| FIQ（快速中断） | FIQ | 0x0000001C | 0xFFFF001C |

# ARM处理器处理异常的具体步骤

**NO.1**

- 将下一条指令的地址存入相应链接寄存器LR，以便程序在处理异常返回时能从正确的位置重新开始执行。

**NO.2**

- 将CPSR复制到相应的SPSR中。

**NO.3**

- 根据异常类型，强制设置CPSR的运行模式位。

**NO.4**

- 强制PC从相关的异常向量地址取下一条指令执行，从而跳转到相应的异常处理程序处。同时设置中断禁止位。

# ARM处理器处理异常的具体步骤

NO.1

- 将下一条指令的地址存入相应链接寄存器LR，以便程序在处理异常返回时能从正确的位置重新开始执行。

| | |
|---|---|
| ARM状态 | • LR寄存器中保存的是下一条指令的地址（当前执行指令的地址+4/8，与异常类型有关）。 |
| Thumb状态 | • LR寄存器中保存当前执行指令地址偏移一个值后的地址。（△） |

★注意：若当前处理器处于Thumb状态，则当异常向量地址加载入PC时，处理器会自动切换到ARM状态。

# 异常响应的伪代码

硬件自动完成

R14_<Exception_Mode> = Return Link

SPSR_<Exception_Mode> = CPSR

CPSR[4:0] = Exception Mode Number

CPSR[5] = 0

If <Exception_Mode> == Reset or FIQ then

{CPSR[6] = 1}

CPSR[7] = 1

PC = Exception Vector Address

# 异常响应的伪代码

硬件自动完成

R14_<Exception_Mode> = Return Link

SPSR_<Exce

CPSR[4:0] =

CPSR[5] = 0

If <Exception

{CPSR[6] = 1

CPSR[7] = 1

PC = Exception Vector Address

在ARM状态下执行

禁止快速中断

禁止正常中断

转入异常入口地址

# ARM处理器异常返回的具体步骤

**NO.1**

- 将链接寄存器 LR 的值减去相应的偏移量后送到PC中。

**NO.2**

- 将SPSR复制回CPSR中。(※)

★注意：恢复 CPSR 的动作会将T、F和I位自动恢复为异常发生前的值。

**NO.3**

- 若在进入异常处理时设置了中断禁止位，要在此清除。

**NO.4**

- 可以认为应用程序总是从复位异常处理程序开始执行的，因此复位异常处理程序不需要返回。

## 进入/退出异常处理的部分环境

| 异常 | 返回指令 | R14_mode保存的地址 | |
|---|---|---|---|
| | | ARM 状态 | Thumb 状态 |
| BL(※) | MOV PC，R14 | PC－4 | PC－2 |
| SWI | MOVS PC，R14_svc | PC－4 | PC－2 |
| UDEF | MOVS PC，R14_und | PC－4 | PC－2 |
| FIQ | SUBS PC，R14_fiq，＃4 | PC’－4(=PC) | 同左 |
| IRQ | SUBS PC，R14_irq，＃4 | PC’－4(=PC) | 同左 |
| PABT | SUBS PC，R14_abt，＃4 | PC－4 | 同左 |
| DABT | SUBS PC，R14_abt，＃8 | PC’－4(=PC) | 同左 |
| RESET | NA | — | — |

异常返回后需要重新执行指令

# 各类异常的具体描述

## 复位

- ✓ 一旦有复位信号输入，ARM处理器立刻停止执行当前指令。
- ✓ 复位后，ARM处理器在禁止中断的管理模式下，从地址0x00000000或0xFFFF0000开始执行指令。

# 各类异常的具体描述

## 未定义指令异常

✓ 当ARM处理器执行协处理器指令时，它必须等待任一外部协处理器应答后，才能真正执行这条指令。若协处理器没有响应，就会出现未定义指令异常。

✓ 可用于在没有物理协处理器（硬件）的系统上，对协处理器进行软件仿真，或在软件仿真时进行指令扩展。

✓ 无论处于ARM状态还是Thumb状态，处理器均可通过执行指令MOVS PC,R14_und，返回到未定义指令的下一条指令。

从R14_und恢复PC +
从SPSR_und恢复CPSR

# 各类异常的具体描述

## 软件中断异常

- ✓ 由执行SWI指令产生，可使用此机制进行软件仿真。
- ✓ 无论处于ARM状态还是Thumb状态，处理器均可通过执行指令MOVS PC,R14_svc，返回到SWI指令的下一条指令。

从R14_svc恢复PC +
从SPSR_svc恢复CPSR

# 各类异常的具体描述

## 指令预取中止（取指令存储器中止）

✓ 若处理器预取指令的地址不存在，或该地址不允许当前指令访问，则存储器会向处理器发出中止信号。注意：仅当预取的指令被执行时，才会产生指令预取中止异常。

## 数据中止（数据访问存储器中止）

✓ 若处理器数据访问指令的地址不存在，或该地址不允许当前指令访问时，产生数据中止异常。

※无论Thumb状态还是ARM状态，从中止模式返回的指令：

```
SUBS PC, R14_abt, #4      ; 指令预取中止
SUBS PC, R14_abt, #8      ; 数据访问中止
```

# 各类异常的具体描述

## IRQ异常

✓ 属于正常的中断请求，系统外设可通过该异常请求中断服务。 S3C2440(基于ARM920T内核)：EINT0~EINT23

✓ 触发条件：当处理器的外部中断请求引脚n_IRQ有效，且CPSR中的I=0。

※无论Thumb状态还是ARM状态，从IRQ模式返回的指令：

SUBS PC, R14_irq, #4

# 各类异常的具体描述

## FIQ异常

✓ 触发条件：当处理器的外部中断请求引脚n_FIQ有效，且CPSR中的F=0。

※无论Thumb状态还是ARM状态，从FIQ模式返回的指令：

SUBS PC, R14_fiq, #4

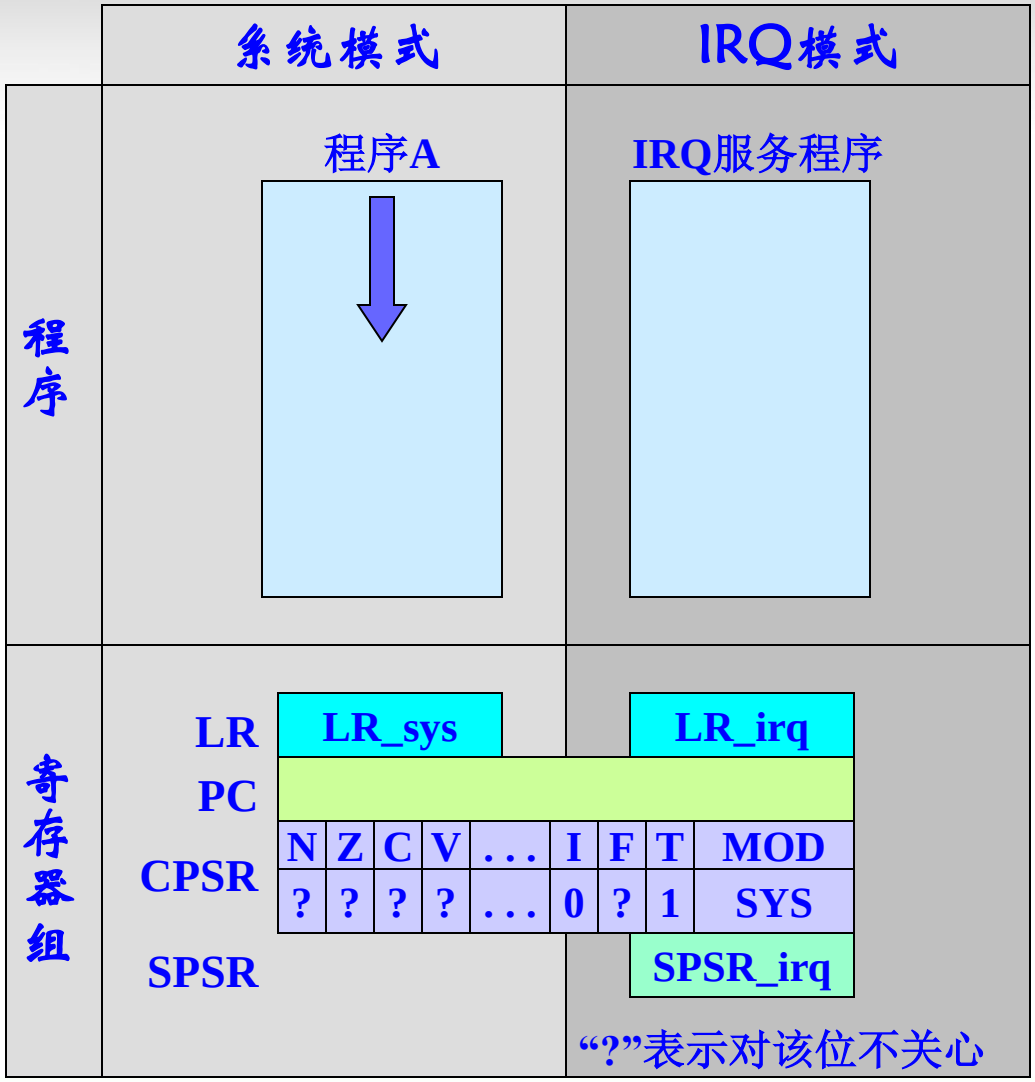
✓ 支持数据传送和通道处理，有足够的私有寄存器，因而在应用中可避免对寄存器保存的需求，减少了开销。

## 异常处理总结

| 异常类型 | 异常返回的位置 | 需设置的 PC 值 | 对应的返回指令 |
|---|---|---|---|
| 复位 | 无返回 | 无 | 无 |
| 未定义指令 | 未定义指令后第 1 条指令地址 | LR | MOVS PC, LR |
| 软中断 SWI | SWI 指令后第 1 条指令地址 | LR | MOVS PC, LR |
| 指令预取中止 | 本预取中止指令地址 | LR-4 | SUBS PC, LR, #4 |
| IRQ | 断点后第 1 条指令地址 | LR-4 | SUBS PC, LR, #4 |
| FIQ | 断点后第 1 条指令地址 | LR-4 | SUBS PC, LR, #4 |
| 数据预取中止 | 本数据中止指令地址 | LR-8 | SUBS PC, LR, #8 |

# 图示进入异常过程

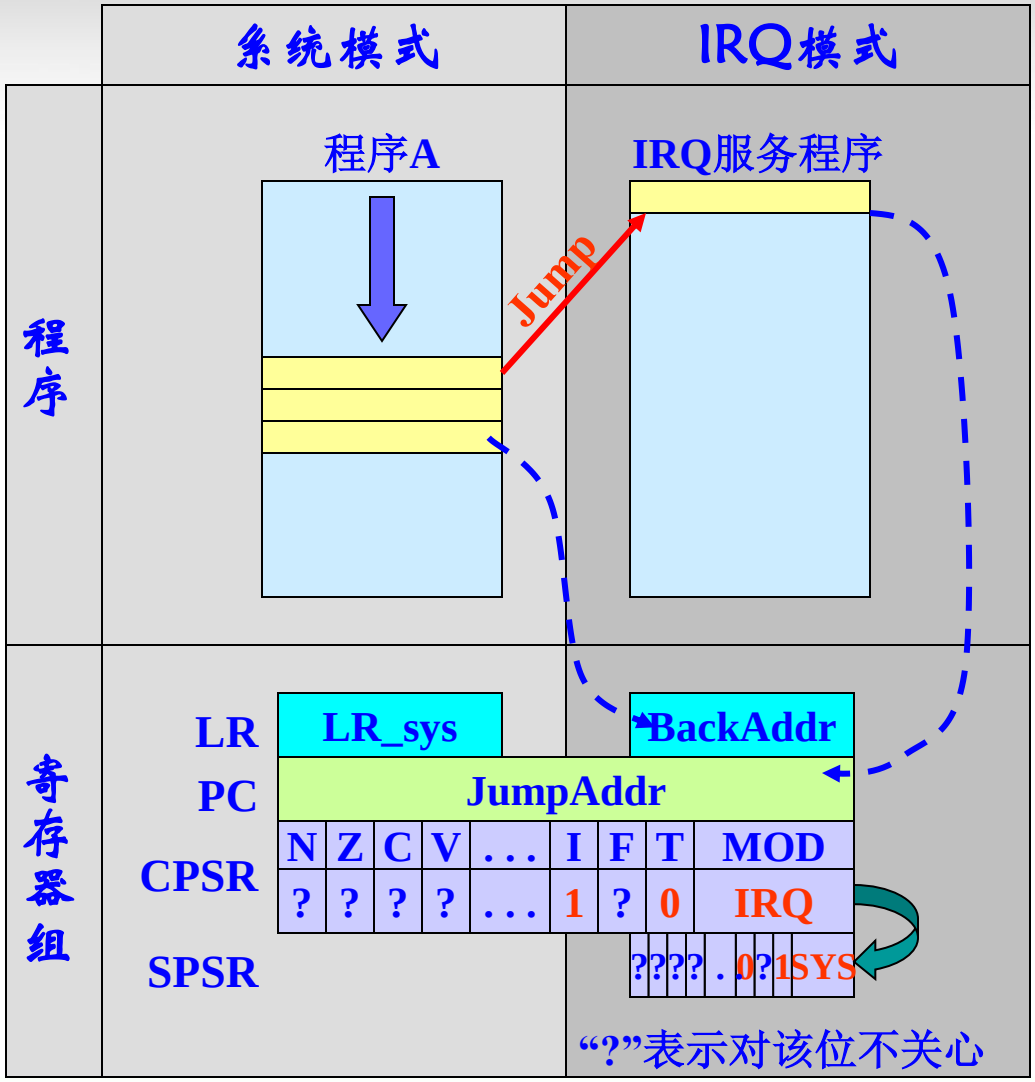
1. 程序在系统模式下运行用户程序，假定当前处理器状态为Thumb状态、允许IRQ中断；



# 图示进入异常过程

2. 用户程序运行时发生IRQ中断，硬件完成以下动作：

- 将CPSR寄存器内容存入IRQ模式的SPSR寄存器
- 置位I位（禁止IRQ中断）
- 清零T位（进入ARM状态）
- 设置MOD位，切换处理器模式至IRQ模式
- 将第三条指令的地址(PC)存入IRQ模式的LR寄存器
- 将跳转地址放入PC，实现跳转

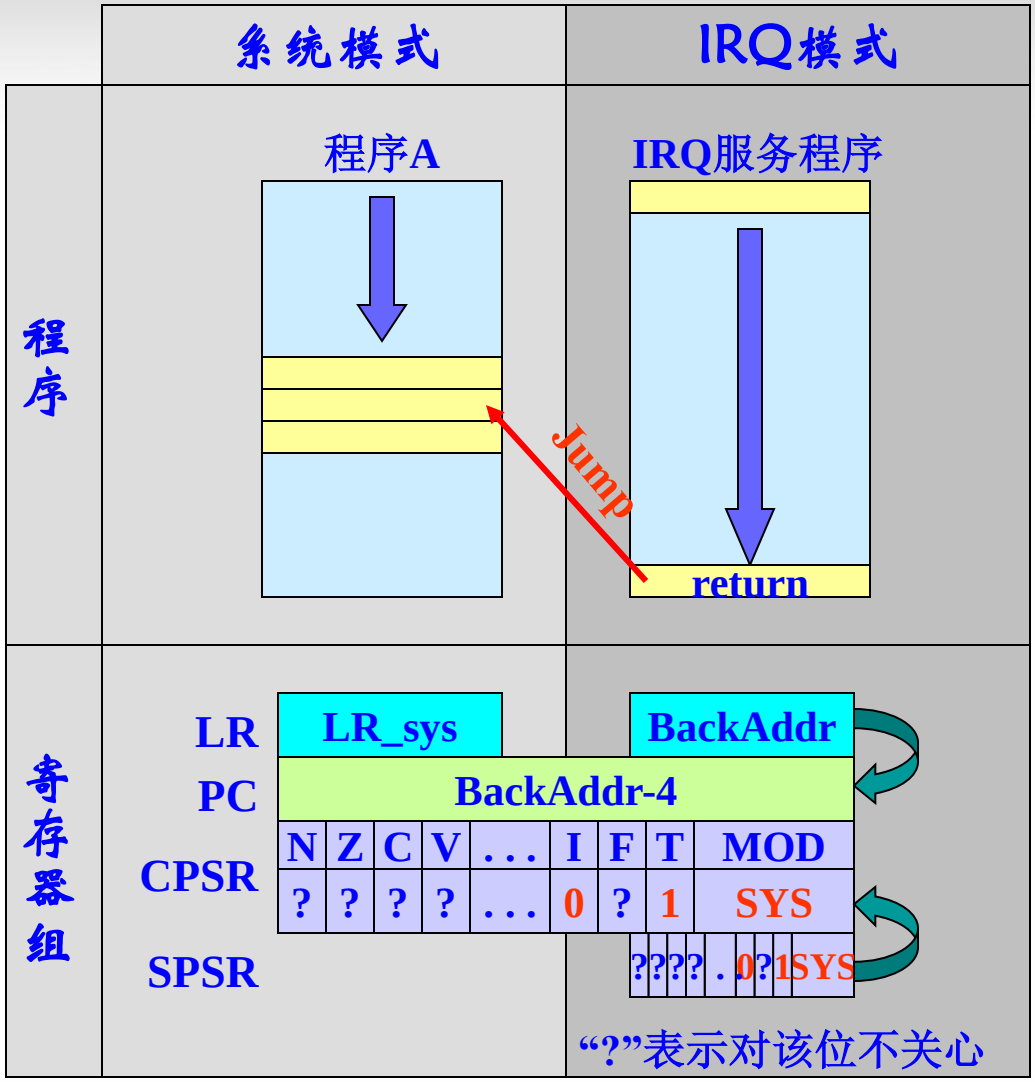


# 图示退出异常过程



在异常处理结束后，异常处理程序完成以下动作：

- 将SPSR寄存器的值复制回CPSR寄存器；
- 将LR寄存的值减去一个常量(IRQ是4)后放入PC，从而返回到被中断的用户程序继续执行。



# The End!

# 本章要点

- ARM微处理器的结构。
  - ARM微处理器的内核结构。
  - ARM微处理器的寄存器组织。
- ARM微处理器的运行模式与工作状态。
  - 七种运行模式的特点。
  - 两种工作状态的特点及相互转换方法。
- ARM微处理器的存储格式。
  - 两种存储器数据格式的区别。
  - 分页式存储管理及对应的地址转换方法。
- ARM微处理器的异常处理。
  - 异常向量、异常向量地址、异常向量表等概念。
  - 七种异常类型及其异常处理过程。